arXiv:2512.24651v1 [cs.RO] 31 Dec 2025

HYBRID MOTION PLANNING WITH DEEP REINFORCEMENT LEARNING FOR MOBILE ROBOT NAVIGATION

Yury Kolomeytsev
Department of Computational Mathematics and Cybernetics
Lomonosov Moscow State University
yury.kolomeytsev@gmail.com

Dmitry Golembiovsky
Department of Computational Mathematics and Cybernetics
Lomonosov Moscow State University

ABSTRACT

Autonomous mobile robots operating in complex, dynamic environments face the dual challenge of navigating large-scale, structurally diverse spaces with static obstacles while safely interacting with various moving agents. Traditional graph-based planners excel at long-range pathfinding but lack reactivity, while Deep Reinforcement Learning (DRL) methods demonstrate strong collision avoidance but often fail to reach distant goals due to a lack of global context. We propose *Hybrid Motion Planning with Deep Reinforcement Learning* (HMP-DRL), a hybrid framework that bridges this gap. Our approach utilizes a graph-based global planner to generate a path, which is integrated into a local DRL policy via a sequence of checkpoints encoded in both the state space and reward function. To ensure social compliance, the local planner employs an entity-aware reward structure that dynamically adjusts safety margins and penalties based on the semantic type of surrounding agents. We validate the proposed method through extensive testing in a realistic simulation environment derived from real-world map data. Comprehensive experiments demonstrate that HMP-DRL consistently outperforms other methods, including state-of-the-art approaches, in terms of key metrics of robot navigation: success rate, collision rate, and time to reach the goal. Overall, these findings confirm that integrating long-term path guidance with semantically-aware local control significantly enhances both the safety and reliability of autonomous navigation in complex human-centric settings.

Keywords Robotics · Reinforcement learning · Motion planning · Path planning · Collision avoidance · Deep learning · Robot navigation

1 Introduction

Mobile robots are rapidly transitioning from controlled industrial settings to complex, human-centric environments. Robots designed to assist humans are increasingly being deployed across a wide range of sectors, including manufacturing and industrial automation, agriculture, space exploration, healthcare, domestic assistance, logistics and delivery, and facility and public-space services [1–3]. Autonomous mobile robots combine a mobile platform, sensors, and onboard intelligence to adapt in real time to dynamic environments, rather than depending on fixed infrastructure like automated guided vehicles (AGVs). A central capability underlying these systems is autonomous navigation, which allows a robot to operate independently, perceive the environment, and act safely to achieve specified goals without operator supervision or control.

Typical applications of autonomous robots include material handling in warehouses and factories; hospital logistics (e.g., transporting laboratory samples, blood components, and medications); service robots in

5202 ceD 13]OR.sc[1v15642.2152:viXra

具有深度强化学习的混合运动规划用于移动机器人导航

Yury Kolomeytsev 莫斯科国立罗蒙诺索夫大学计算数学与控制论系 yury.kolomeytsev@gmail.com

Dmitry Golembiovsky 莫斯科国立罗蒙诺索夫大学计算数学与控制论系

抽象的

在复杂、动态环境中运行的自主移动机器人面临着双重挑战，即在具有静态障碍物的大规模、结构多样的空间中导航，同时与各种移动主体安全地交互。传统的基于图的规划器擅长远程寻路，但缺乏反应性，而深度强化学习（DRL）方法表现出强大的碰撞避免能力，但由于缺乏全局背景，往往无法达到遥远的目标。我们提出 *Hybrid Motion Planning with Deep Reinforcement Learning* (HMP-DRL)，这是一个弥补这一差距的混合框架。我们的方法利用基于图的全局规划器来生成一条路径，该路径通过状态空间和奖励函数中编码的一系列检查点集成到本地 DRL 策略中。为了确保社会合规性，本地规划者采用实体感知奖励结构，根据周围代理的语义类型动态调整安全边际和惩罚。我们通过在从真实世界地图数据派生的真实模拟环境中进行广泛测试来验证所提出的方法。综合实验表明，在机器人导航的关键指标（成功率、碰撞率和到达目标的时间）方面，HMP-DRL 始终优于其他方法，包括最先进的方法。总的来说，这些发现证实，将长期路径引导与语义感知的本地控制相结合，可以显着提高复杂的以人为中心的环境中自主导航的安全性和可靠性。

Keywords 机器人 · 强化学习 · 运动规划 · 路径规划 · 防撞 · 深度学习 · 机器人导航

1 简介

移动机器人正在迅速从受控工业环境过渡到复杂的、以人为中心的环境。旨在帮助人类的机器人越来越多地部署在各个领域，包括制造和工业自动化、农业、太空探索、医疗保健、家庭援助、物流和交付以及设施和公共空间服务[1-3]。自主移动机器人结合了移动平台、传感器和机载智能，可以实时适应动态环境，而不是依赖于自动导引车 (AGV) 等固定基础设施。这些系统的核心功能是自主导航，它允许机器人独立操作、感知环境并安全行动以实现指定目标，而无需操作员监督或控制。

自主机器人的典型应用包括仓库和工厂的物料搬运；医院后勤（例如，运输实验室样本、血液成分和药物）；服务机器人在

commercial buildings and hotels for indoor delivery and visitor guidance; autonomous cleaning; security patrol and inspection; and last-mile urban delivery of groceries, meals, and parcels [3–5].

The core challenge in such deployments is twofold: traversing large-scale, structurally complex environments (global planning) while safely interacting with diverse dynamic agents (local planning and collisions avoidance). Consider a sidewalk delivery robot [6] operating in an urban setting: it must travel up to several kilometers, traversing diverse surfaces such as sidewalks and crosswalks, while accounting for static obstacles like buildings, bus stops, flowerbeds and curbs. Traditional search-based planners (e.g., A*, Dijkstra) excel at finding optimal paths over these long distances using static maps but struggle to react fluidly to unmapped, moving entities.

Conversely, Deep Reinforcement Learning (DRL) methods have demonstrated superior performance in local collision avoidance and crowd navigation [7–9]. However, pure DRL approaches typically lack global context. When applied to large environments with complex geometry, they often fail to reach distant goals, getting trapped in local minima due to the absence of global guidance.

Furthermore, safe navigation requires more than just treating obstacles as generic geometric shapes. A robot must navigate in a socially compliant manner, accounting for the semantic diversity of the environmental entities around it. A child, a bicyclist, an adult, and a concrete wall all impose different safety constraints and motion patterns. Treating a vulnerable pedestrian with the same safety margin as a static obstacle is insufficient for socially compliant navigation.

To address these challenges, we introduce *Hybrid Motion Planning with Deep Reinforcement Learning* (HMP-DRL), a novel hybrid method that bridges the gap between global path planning and local motion planning with entity-aware collision avoidance. Our approach leverages a global path planner to provide long-term guidance through complex static environments (handling curbs, walls, and restricted zones), while a local DRL policy handles immediate interaction with dynamic agents and static entities. This decomposition allows the robot to navigate kilometers without losing its way, while simultaneously exhibiting sophisticated, entity-aware collision avoidance to ensure safe navigation.

Our main contributions are as follows:

- We propose a hybrid motion planning method that combines graph-based global search with deep RL-based local planning. This hybrid structure enables the robot to utilize map information for reaching distant goals while maintaining the reactivity required for dynamic obstacle avoidance.
- We design a reward function that encourages the robot to follow the global path and minimize travel time. The function uses semantic awareness by penalizing the robot differently depending on the type of agent or obstacle it encounters. Specifically, the required safe distance and collision penalties are dynamically adjusted based on the entity type, resulting in safer and more socially compliant behavior.
- We introduce a training and testing environment designed to mimic realistic outdoor settings with various static and dynamic obstacles. Our experimental results demonstrate that this new approach consistently outperforms standard navigation techniques, including state-of-the-art methods.

2 Related Work

Robot navigation research has evolved from traditional rule-based methods and classical planning algorithms to learning-based approaches, with recent work focusing on crowd navigation and hybrid architectures that combine global and local planning.

Classical approaches such as the Social Force Model (SFM) [10] and Optimal Reciprocal Collision Avoidance (ORCA) [11] use hand-crafted interaction rules for collision avoidance. SFM applies attractive and repulsive forces but is prone to local minima; ORCA uses velocity constraints under reciprocal assumptions, expecting all agents to cooperate equally in avoiding collisions—an assumption rarely met in real-world settings with unpredictable pedestrians. Both methods can produce short-sighted or unnatural behavior and struggle to generalize across diverse social environments.

商业建筑和酒店的室内送货和访客引导；自主清洁；保安巡逻、检查；杂货、膳食和包裹的最后一英里城市配送[3-5]。
此类部署的核心挑战是双重的：穿越大规模、结构复杂的环境（全局规划），同时与不同的动态代理安全交互（局部规划和避免碰撞）。考虑在城市环境中运行的人行道送货机器人 [6]：它必须行驶长达几公里，穿越人行道和人行横道等不同的表面，同时考虑建筑物、公交车站、花坛和路缘石等静态障碍物。传统的基于搜索的规划器（例如，A*、Dijkstra）擅长使用静态地图在这些长距离上找到最佳路径，但很难对未映射的移动实体做出流畅的反应。

相反，深度强化学习（DRL）方法在局部防撞和人群导航方面表现出了卓越的性能[7-9]。然而，纯粹的 DRL 方法通常缺乏全局背景。当应用于具有复杂几何形状的大型环境时，它们通常无法达到遥远的目标，由于缺乏全局指导而陷入局部极小值。

此外，安全导航需要的不仅仅是将障碍物视为通用几何形状。机器人必须以符合社会规范的方式进行导航，并考虑到周围环境实体的语义多样性。儿童、骑自行车的人、成人和混凝土墙都会施加不同的安全约束和运动模式。以与静态障碍物相同的安全裕度对待易受伤害的行人不足以实现符合社会要求的导航。

为了应对这些挑战，我们引入了 *Hybrid Motion Planning with Deep Reinforcement Learning* (HMP-DRL)，这是一种新颖的混合方法，它弥合了全局路径规划和具有实体感知碰撞避免的局部运动规划之间的差距。我们的方法利用全局路径规划器在复杂的静态环境中提供长期指导（处理路缘石、墙壁和限制区域），而本地 DRL 策略则处理与动态代理和静态实体的即时交互。这种分解使机器人能够在不迷路的情况下导航数公里，同时展示复杂的、实体感知的防撞功能，以确保安全导航。

我们的主要贡献如下：

- 我们提出了一种混合运动规划方法，它将基于图的全局搜索与基于深度强化学习的局部规划相结合。这种混合结构使机器人能够利用地图信息到达远处的目标，同时保持动态避障所需的反应性。
- 我们设计了一个奖励函数，鼓励机器人遵循全局路径并最大限度地减少旅行时间。该功能使用语义意识，根据机器人遇到的代理或障碍物的类型对机器人进行不同的惩罚。具体来说，所需的安全距离和碰撞惩罚会根据实体类型动态调整，从而实现更安全、更符合社会规范的行为。
- 我们引入了一个训练和测试环境，旨在模拟具有各种静态和动态障碍的真实户外环境。我们的实验结果表明，这种新方法始终优于标准导航技术，包括最先进的方法。

2 相关工作

机器人导航研究已经从传统的基于规则的方法和经典规划算法发展到基于学习的方法，最近的工作重点是人群导航和结合全局和局部规划的混合架构。

社会力模型 (SFM) [10] 和最优相互碰撞避免 (ORCA) [11] 等经典方法使用手工制定的交互规则来避免碰撞。 SFM 应用吸引力和排斥力，但容易出现局部极小值； ORCA 在相互假设下使用速度约束，期望所有智能体平等合作以避免碰撞——这种假设在行人不可预测的现实环境中很少遇到。这两种方法都会产生短视或不自然的行为，并且很难在不同的社会环境中推广。

Learning-based methods have gained significant attention as alternatives to hand-crafted rules. Imitation learning approaches derive navigation strategies from demonstrations of desired behaviors. For example, [12] combines behavior cloning with generative adversarial imitation learning (GAIL) to develop a policy that operates directly on raw depth images, bypassing explicit pedestrian tracking. [13] applies maximum entropy inverse RL to model human navigation as a mixture distribution capturing both discrete decisions (e.g., passing left or right) and continuous trajectory variance. However, these methods depend heavily on the quality and scale of demonstrations, which can be resource-intensive to collect and may limit generalization to novel scenarios.

Deep reinforcement learning (DRL) offers an alternative approach by treating navigation as a sequential decision-making problem. CADRL [7] formulates decentralized multi-agent collision avoidance by training a value network that estimates time-to-goal from the joint state of the robot and its neighbors, achieving faster navigation than ORCA while remaining real-time implementable. SA-CADRL [14] extends this idea with norm-inducing rewards that penalize violations of social conventions (e.g., passing on the wrong side) and a symmetric network architecture with weight-sharing and max-pooling to handle variable numbers of agents, trading some efficiency for socially compliant behavior. To better scale to dense crowds, LSTM-RL [15] uses recurrent networks to encode an arbitrary number of neighboring agents, significantly reducing failure rates in crowded scenarios. SARL [8] further improves crowd modeling through an attention-based value network that jointly reasons about human-robot and human-human interactions, enhancing anticipation of crowd dynamics. More recently, [16] proposes a spatial-temporal value network and a hazard-aware reward structure that explicitly reasons about pedestrian speeds and hazardous regions to improve safety and comfort. Recently, EB-CADRL [9] introduced entity-type awareness into the state representation and reward function, enabling differentiated safety margins for various agent types. The method demonstrated superior safety for vulnerable agents in dense crowds and achieved significant training speedups through parallelized experience collection.

A significant limitation of pure DRL methods is that they focus primarily on local collision avoidance and are typically trained and evaluated in small, bounded environments. When applied to large-scale maps or long-range navigation tasks, these methods fail to reach distant goals, often getting stuck or exhibiting suboptimal behavior due to the absence of global path guidance. Furthermore, they neglect static environmental structures such as buildings, walls, and terrain boundaries. To bridge this gap, recent research has explored hybrid approaches that combine classical path planning with learning-based local control.

DRL-based hybrid methods integrate global planners with learned local policies. SARL* [17] extends SARL by using Dijkstra's algorithm for dynamic local goal setting and a map-based safe action space; however, it rebuilds paths every time step (computationally inefficient) and relies on rule-based action filtering rather than learned obstacle avoidance. G2RL [18] incorporates a static A* path as a guidance channel into the DRL state space, but the fixed global path becomes problematic when obstacles block the path, and the coarse discretization (4-directional actions on 200×200 grids) limits real-world applicability. Similarly, [19] integrates A* with a DQN agent for automated driving, demonstrating faster convergence, but lacks pedestrian awareness and uses discrete actions resulting in jerky control. GDAE [20] combines frontier-based waypoint selection with TD3 for unknown environments but does not utilize a global path and lacks dynamic obstacle handling. VDPF-TD3 [21] uses TD3 to optimize the balance between modified potential field forces, effectively escaping local optima, but relies on geometric obstacle representations and lacks real-world validation.

Classical hybrid planners combine global path planning with reactive local control. Zhong et al. [22] propose Safe A* with risk-aware costs combined with an Adaptive Window Approach, while IAB-A* [23] introduces bidirectional search with B-spline smoothing integrated with DWA. Both methods achieve computational efficiency but depend heavily on manually tuned parameters and do not differentiate between obstacle types. The RC-Map approach [24] uses rectangular decomposition for fast topological planning (10ms vs 740ms for grid methods) but produces sub-optimal paths and struggles with non-orthogonal environments.

Sampling-based hybrids improve path search efficiency. RRTstar-RL [25] replaces uniform RRT* sampling with a learned probability distribution, achieving 28% shorter paths and 2x speedup, but requires extensive training data and lacks mechanisms for dynamic obstacle avoidance.

基于学习的方法作为手工制定规则的替代方案而受到广泛关注。模仿学习方法从期望行为的演示中得出导航策略。例如，[12]将行为克隆与生成对抗性模仿学习（GAIL）相结合，开发出一种直接对原始深度图像进行操作的策略，绕过了显式的行人跟踪。 [13] 应用最大熵逆 RL 将人类导航建模为捕获离散决策（例如，向左或向右通过）和连续轨迹方差的混合分布。然而，这些方法在很大程度上取决于演示的质量和规模，收集这些方法可能需要消耗大量资源，并且可能会限制对新场景的推广。

深度强化学习（DRL）提供了一种替代方法，将导航视为顺序决策问题。 CADRL [7] 通过训练一个价值网络来制定分散式多智能体碰撞避免，该价值网络根据机器人及其邻居的联合状态估计到达目标的时间，实现比 ORCA 更快的导航，同时保持实时可实现性。 SA-CADRL [14] 通过规范诱导奖励来扩展这一想法，这些奖励可以惩罚违反社会惯例的行为（例如，在错误的一方传递），以及具有权重共享和最大池化的对称网络架构来处理可变数量的代理，以牺牲一些效率来换取符合社会规范的行为。为了更好地扩展到密集人群，LSTM-RL [15] 使用循环网络对任意数量的相邻代理进行编码，从而显着降低拥挤场景中的失败率。 SARL [8] 通过基于注意力的价值网络进一步改进人群建模，该网络共同推理人与机器人和人与人的交互，增强对人群动态的预期。最近，[16]提出了一个时空价值网络和一个危险感知奖励结构，它明确地推理行人速度和危险区域，以提高安全性和舒适度。最近，EB-CADRL [9]将实体类型感知引入状态表示和奖励函数中，为各种代理类型提供了差异化的安全边际。该方法证明了密集人群中易受攻击的代理具有卓越的安全性，并通过并行经验收集实现了显着的训练加速。

纯 DRL 方法的一个显着限制是它们主要关注局部碰撞避免，并且通常在小型有界环境中进行训练和评估。当应用于大规模地图或远程导航任务时，这些方法无法达到遥远的目标，常常由于缺乏全局路径引导而陷入困境或表现出次优行为。此外，他们忽略了静态环境结构，例如建筑物、墙壁和地形边界。为了弥补这一差距，最近的研究探索了将经典路径规划与基于学习的局部控制相结合的混合方法。
基于 DRL 的混合方法将全球规划者与学习到的本地政策相结合。 SARL* [17] 通过使用 Dijkstra 算法进行动态局部目标设置和基于地图的安全行动空间来扩展 SARL；然而，它在每个时间步都会重建路径（计算效率低下），并且依赖于基于规则的动作过滤而不是学习的避障。 G2RL [18] 将静态 A* 路径作为引导通道纳入 DRL 状态空间，但当障碍物阻挡路径时，固定的全局路径就会出现问题，并且粗离散化（200 × 200 网格上的 4 向动作）限制了现实世界的适用性。同样，[19] 将 A* 与 DQN 代理集成用于自动驾驶，展示了更快的收敛速度，但缺乏行人意识并使用离散动作，导致控制不稳定。 GDAE [20] 将基于边界的航路点选择与 TD3 结合起来用于未知环境，但不利用全局路径并且缺乏动态障碍物处理。 VDPF-TD3[21]使用TD3来优化修改后的势场力之间的平衡，有效地逃避局部最优，但依赖于几何障碍物表示并且缺乏现实世界的验证。

经典的混合规划器将全局路径规划与反应性局部控制相结合。钟等人。 [22] 提出了具有风险意识成本的 Safe A* 与自适应窗口方法相结合，而 IAB-A* [23] 则引入了与 DWA 集成的 B 样条平滑的双向搜索。这两种方法都实现了计算效率，但严重依赖于手动调整的参数，并且不区分障碍物类型。 RC-Map 方法 [24] 使用矩形分解进行快速拓扑规划（网格方法为 10 毫秒 vs 740 毫秒），但会产生次优路径并且难以应对非正交环境。
基于采样的混合提高了路径搜索效率。 RRTstar-RL [25] 用学习的概率分布取代了均匀的 RRT* 采样，实现了 28% 的短路径和 2 倍的加速，但需要大量的训练数据，并且缺乏动态避障机制。

Our HMP-DRL approach addresses these limitations by (i) computing a single global path per episode and using its checkpoints in both the reward and the network input to provide path guidance at low cost, (ii) learning collision avoidance directly through the reward with a fine-grained discrete action space that still enables smooth motion, and (iii) encoding entity types in the state and reward to achieve socially compliant navigation with type-dependent safety margins.

3 Problem Formulation

We address the problem of autonomous robot navigation towards a goal in dynamic environments populated with various entities, including dynamic agents and static obstacles. This problem can be formulated as a Markov decision process (MDP). The robot must navigate in a socially compliant manner, avoiding collisions while reaching its goal. We define the state of each agent (robot or environmental entity) in terms of observable and unobservable components:

- Observable states: position $\mathbf{p} = [p_x, p_y]$, velocity $\mathbf{v} = [v_x, v_y]$, and radius r.
- Unobservable states: goal position $\mathbf{g} = [g_x, g_y]$, preferred speed v_{pref}, and heading angle θ.

While acting in the environment, the robot knows its observable and unobservable states, as well as the observable states of all entities. We use the robot-centric parameterization described in [7] and [15], where the robot is positioned at the origin and the x-axis is aligned with the robot's goal direction. Denote e_i as a type of entity i. The robot's state and the i-th entity's observable state at time t could be represented as:

$$\begin{aligned} \mathbf{s}_t^r &= [d_g, v_x, v_y, r, v_{\text{pref}}, \theta], \\ \mathbf{s}_t^{io} &= [p_x^i, p_y^i, v_x^i, v_y^i, r^i, d^i, r^i + r, e_i], \end{aligned} \tag{1}$$

where $d_g = \|\mathbf{p}_t - \mathbf{g}\|_2$ is the distance from the robot to the goal, and $d^i = \|\mathbf{p}_t - \mathbf{p}_t^i\|_2$ is the distance from the robot to the i-th entity.

The robot can acquire its own state and the observable states of other agents at each time step. The joint state at time t is defined by:

$$\mathbf{s}_t^j = [\mathbf{s}_t^r, \mathbf{s}_t^{1o}, \mathbf{s}_t^{2o}, \ldots, \mathbf{s}_t^{no}]. \tag{2}$$

The robot's velocity $\mathbf{v}_t$ is determined by the action command $\mathbf{a}_t$ from the navigation policy, i.e., $\mathbf{v}_t = \mathbf{a}_t = \pi(\mathbf{s}_t^j)$.

The reward function $R(\mathbf{s}_t^j, \mathbf{a}_t)$ is defined as a mapping from the current state $\mathbf{s}_t^j$ and action $\mathbf{a}_t$ to a scalar value, which represents the immediate benefit of taking action $\mathbf{a}_t$ in state $\mathbf{s}_t^j$. Formally:

$$R(\mathbf{s}_t^j, \mathbf{a}_t) = r_t, \tag{3}$$

where r_t is the reward received at time step t.

Our objective is to find the optimal policy $\pi^*(\mathbf{s}_t^j)$ that maximizes the expected reward.

The optimal value function is given by [26]:

$$V^*(\mathbf{s}_t^j) = \sum_{\tilde{t}=t}^{T} \gamma^{\tilde{t} \cdot v_{\text{pref}}} R_{\tilde{t}}(\mathbf{s}_{\tilde{t}}^j, \pi^*(\mathbf{s}_{\tilde{t}}^j)), \tag{4}$$

where $\gamma \in (0, 1)$ is a discount factor and T is the time step at which the episode ends.

Using value iteration, the optimal policy is derived as:

$$\begin{aligned} \pi^*\left(\mathbf{s}_t^j\right) = \arg\max_{\mathbf{a}_t \in \mathbf{A}} \Bigg[& R\left(\mathbf{s}_t^j, \mathbf{a}_t\right) + \gamma^{\Delta t \cdot v_{\text{pref}}} \cdot \\ & \int_{\mathbf{s}_{t+\Delta t}^j} P\left(\mathbf{s}_{t+\Delta t}^j \mid \mathbf{s}_t^j, \mathbf{a}_t\right) V^*\left(\mathbf{s}_{t+\Delta t}^j\right) d\mathbf{s}_{t+\Delta t}^j \Bigg], \end{aligned} \tag{5}$$

我们的 HMP-DRL 方法通过以下方式解决了这些限制：(i) 计算每集的单个全局路径，并在奖励和网络输入中使用其检查点以低成本提供路径指导；(ii) 通过奖励直接通过细粒度的离散动作空间学习避免碰撞，该空间仍可实现平滑运动；(iii) 对状态和奖励中的实体类型进行编码，以实现具有依赖于类型的安全裕度的社会兼容导航。

3 问题表述

我们解决了机器人在充满各种实体（包括动态代理和静态障碍物）的动态环境中实现目标的自主导航问题。这个问题可以表述为马尔可夫决策过程（MDP）。机器人必须以符合社会规范的方式导航，在达到目标的同时避免碰撞。我们根据可观察和不可观察的组件来定义每个代理（机器人或环境实体）的状态：

- Observable states: position $\mathbf{p} = [p_x, p_y]$, velocity $\mathbf{v} = [v_x, v_y]$, and radius r.
- Unobservable states: goal position $\mathbf{g} = [g_x, g_y]$, preferred speed v_{pref}, and heading angle θ.

在环境中行动时，机器人知道其可观察和不可观察状态，以及所有实体的可观察状态。我们使用[7]和[15]中描述的以机器人为中心的参数化，其中机器人位于原点，x轴与机器人的目标方向对齐。将 e_i 表示为实体 i 的一种类型。机器人的状态和第 i 个实体在 t 时刻的可观察状态可以表示为：

$$\begin{aligned} \mathbf{s}_t^r &= [d_g, v_x, v_y, r, v_{\text{pref}}, \theta], \\ \mathbf{s}_t^{io} &= [p_x^i, p_y^i, v_x^i, v_y^i, r^i, d^i, r^i + r, e_i], \end{aligned} \tag{1}$$

其中$d_g = \|\mathbf{p}_t - \mathbf{g}\|_2$是从机器人到目标的距离，$d^i = \|\mathbf{p}_t - \mathbf{p}_t^i\|_2$是从机器人到第$i$实体的距离。

机器人可以在每个时间步获取自己的状态和其他智能体的可观察状态。 t 时刻的联合状态定义为：

$$\mathbf{s}_t^j = [\mathbf{s}_t^r, \mathbf{s}_t^{1o}, \mathbf{s}_t^{2o}, \ldots, \mathbf{s}_t^{no}]. \tag{2}$$

机器人的速度 $\mathbf{v}_t$ 由导航策略中的动作命令 $\mathbf{a}_t$ 确定，即 $\mathbf{v}_t = \mathbf{a}_t = \pi(\mathbf{s}_t^j)$。

奖励函数 $R(\mathbf{s}_t^j, \mathbf{a}_t)$ 定义为从当前状态 $\mathbf{s}_t^j$ 和操作 $\mathbf{a}_t$ 到标量值的映射，该标量值表示在状态 $\mathbf{s}_t^j$ 下采取操作 $\mathbf{a}_t$ 的直接好处。正式：

$$R(\mathbf{s}_t^j, \mathbf{a}_t) = r_t, \tag{3}$$

其中 r_t 是在时间步 t 收到的奖励。

我们的目标是找到最大化预期奖励的最优策略 $\pi^*(\mathbf{s}_t^j)$。

最优值函数由[26]给出：

$$V^*(\mathbf{s}_t^j) = \sum_{\tilde{t}=t}^{T} \gamma^{\tilde{t} \cdot v_{\text{pref}}} R_{\tilde{t}}(\mathbf{s}_{\tilde{t}}^j, \pi^*(\mathbf{s}_{\tilde{t}}^j)), \tag{4}$$

其中 $\gamma \in (0, 1)$ 是　折扣因子和 T 是情节的时间步长　　e 结束。

使用值迭代，得出最优策略为：

$$\begin{aligned} \pi^*\left(\mathbf{s}_t^j\right) = \arg\max_{\mathbf{a}_t \in \mathbf{A}} \Bigg[& R\left(\mathbf{s}_t^j, \mathbf{a}_t\right) + \gamma^{\Delta t \cdot v_{\text{pref}}} \cdot \\ & \int_{\mathbf{s}_{t+\Delta t}^j} P\left(\mathbf{s}_{t+\Delta t}^j \mid \mathbf{s}_t^j, \mathbf{a}_t\right) V^*\left(\mathbf{s}_{t+\Delta t}^j\right) d\mathbf{s}_{t+\Delta t}^j \Bigg], \end{aligned} \tag{5}$$

where Δt is time step duration, $R(\mathbf{s}_t^j, \mathbf{a}_t)$ is the reward function, $\mathbf{A}$ is the action space, and $P(\mathbf{s}_{t+\Delta t}^j \mid \mathbf{s}_t^j, \mathbf{a}_t)$ is the transition probability.

4 Proposed Approach

In this section, we introduce our new approach which we call Hybrid Motion Planning with Deep Reinforcement Learning (HMP-DRL). HMP-DRL combines a global graph-based path planner with a local deep reinforcement learning policy, enabling long-range navigation through complex environments with static and dynamic obstacles while maintaining entity-aware collision avoidance for safe, socially compliant interaction with dynamic agents.

4.1 Global Planning

Global path planning determines an optimal collision-free path from a starting position to a goal within a known or partially known environment. The core idea is to construct a graph representation of the navigable space and then apply a search algorithm to find the best path through this graph. The global planner considers the map and known static obstacles. Dynamic agents and static obstacles not present on the map are unknown at global planning time but are detected by the robot's sensors during navigation and handled by the local planner in real time.

Graph Construction Methods. A fundamental aspect of global planning is the construction of a graph that captures the connectivity of free space. Different methods can be used to build such graphs, each with distinct advantages:

- *Occupancy Grids.* The environment is discretized into a regular grid of cells, where each cell is marked as occupied or free based on the presence of obstacles. Nodes are placed at the centers of free cells, and edges connect adjacent nodes, enabling grid-based pathfinding algorithms. This method is straightforward, widely used in 2D environments, and integrates seamlessly with sensor data such as occupancy maps.
- *Navigation Meshes.* Free space can be partitioned into a set of connected polygons, most commonly triangles or quadrangles, using cell decomposition methods [27]. Unlike uniform grids, mesh-based representations such as Irregular Triangular Meshes (ITM) and Varying-size ITMs (VITM) can better conform to obstacle boundaries. In particular, VITM enables local refinement in narrow passages while maintaining coarser cells in open areas, providing a balance between geometric fidelity and computational complexity [28]. Quadrangle (QUAD) representations, although less flexible in approximating irregular or oblique obstacle boundaries, offer a simple and structured topology with predictable neighbor relationships. When an inflation layer is applied to account for robot size, QUAD meshes are generally less sensitive than fixed-size triangular meshes, but they may fail to preserve connectivity in narrow or cluttered environments unless a fine resolution is used. In contrast, varying-size triangular meshes demonstrate greater robustness under inflation, particularly in environments with narrow passages, online map updates, or multiple robots with differing dimensions, making them more suitable for such scenarios [29].
- *Voronoi Diagrams.* Given a vectorized map with polygonal obstacle boundaries, a Generalized Voronoi Diagram (GVD) of the free space can be constructed [30, 31]. The GVD edges consist of points equidistant to two or more obstacles, forming a sparse skeleton of the navigable space. The Voronoi vertices (where edges meet) serve as nodes, and the edges themselves serve as paths in the planning graph. Paths computed on Voronoi-based graphs locally maximize clearance from obstacles, improving safety in cluttered or narrow environments, albeit often at the cost of increased path length compared to shortest-path methods.
- *Visibility Graphs.* In environments with polygonal obstacles, nodes can represent the start, goal, and obstacle vertices, with edges connecting mutually visible pairs [32, 33]. This yields the shortest path in terms of Euclidean distance but may produce trajectories that pass too close to obstacle corners.

其中Δt是时间步持续时间，$R(\mathbf{s}_t^j, \mathbf{a}_t)$是奖励函数，$\mathbf{A}$是动作空间，$P(\mathbf{s}_{t+\Delta t}^j \mid \mathbf{s}_t^j, \mathbf{a}_t)$是转移概率。

4 提议的方法

在本节中，我们将介绍我们的新方法，称为深度强化学习混合运动规划（HMP-DRL）。 HMP-DRL 将基于图形的全局路径规划器与本地深度强化学习策略相结合，实现在具有静态和动态障碍的复杂环境中进行远程导航，同时保持实体感知碰撞避免，从而与动态代理进行安全、符合社交要求的交互。

4.1 全局规划

全局路径规划确定已知或部分已知环境中从起始位置到目标的最佳无碰撞路径。核心思想是构建可导航空间的图形表示，然后应用搜索算法来找到通过该图形的最佳路径。全局规划器考虑地图和已知的静态障碍物。地图上不存在的动态代理和静态障碍物在全局规划时是未知的，但在导航过程中由机器人的传感器检测到，并由本地规划器实时处理。

图构造方法。全球规划的一个基本方面是构建一个捕获自由空间连通性的图表。可以使用不同的方法来构建此类图，每种方法都有独特的优点：

- *Occupancy Grids.* 环境被离散化为规则的单元格网格，其中每个单元格根据障碍物的存在被标记为已占用或空闲。节点放置在空闲单元的中心，边缘连接相邻节点，从而实现基于网格的寻路算法。这种方法简单明了，广泛应用于 2D 环境，并与占用地图等传感器数据无缝集成。
- *Navigation Meshes.* 可用空间可以使用单元分解方法划分为一组连接的多边形，最常见的是三角形或四边形[27]。与均匀网格不同，基于网格的表示（例如不规则三角形网格（ITM）和变化尺寸 ITM（VITM））可以更好地符合障碍物边界。特别是，VITM 能够在狭窄通道中进行局部细化，同时在开放区域中保持较粗糙的单元，从而在几何保真度和计算复杂性之间提供平衡[28]。四边形（QUAD）表示虽然在近似不规则或倾斜障碍物边界方面不太灵活，但提供了具有可预测的邻居关系的简单且结构化的拓扑。当应用膨胀层来考虑机器人尺寸时，四边形网格通常不如固定尺寸的三角形网格敏感，但除非使用精细分辨率，否则它们可能无法在狭窄或杂乱的环境中保持连接性。相比之下，不同尺寸的三角形网格在通货膨胀下表现出更大的鲁棒性，特别是在通道狭窄、在线地图更新或具有不同尺寸的多个机器人的环境中，使它们更适合此类场景[29]。
- *Voronoi Diagrams.* 给定一个带有多边形障碍物边界的矢量化地图，可以构建自由空间的广义维诺图（GVD）[30, 31]。 GVD 边缘由与两个或多个障碍物等距的点组成，形成可导航空间的稀疏骨架。 Voronoi 顶点（边相交的地方）充当节点，边本身充当规划图中的路径。基于 Voronoi 的图计算的路径可局部最大化与障碍物的间隙，从而提高杂乱或狭窄环境中的安全性，尽管与最短路径方法相比通常会增加路径长度。 • *Visibility Graphs.* 在具有多边形障碍物的环境中，节点可以表示起点、目标和障碍物顶点，边连接相互可见的对[32, 33]。这会产生欧几里得距离的最短路径，但可能会产生过于靠近障碍角的轨迹。

Algorithm 1 A* Algorithm

Require: occupancy map, start node s, goal node g, neighbor function $\mathcal{N}(\cdot)$, edge cost $c(\cdot,\cdot)$, heuristic $h(\cdot)$
Ensure: shortest path from s to g, if it exists

```
openSet ← {s}, closedSet ← ∅
cameFrom ← empty map
gScore(s) ← 0, fScore(s) ← h(s)
while openSet is not empty do
    n ← arg min_{x∈openSet} fScore(x)
    if n = g then
        return RECONSTRUCTPATH(cameFrom, n)
    end if
    remove n from openSet; add n to closedSet
    for each neighbor m ∈ N(n) do
        if m in closedSet or grid cell m is occupied then
            continue
        end if
        tentative ← gScore(n) + c(n, m)
        if m not in openSet then
            add m to openSet
        else if tentative ≥ gScore(m) then
            continue
        end if
        cameFrom(m) ← n
        gScore(m) ← tentative
        fScore(m) ← gScore(m) + h(m)
    end for
end while
return failure
function RECONSTRUCTPATH(cameFrom, n)
    path ← [n]
    while cameFrom contains n do
        n ← cameFrom(n)
        prepend n to path
    end while
    return path
end function
```

All four graph construction methods are applicable in our HMP-DRL framework; the choice depends on the available map representation and the desired path characteristics. Occupancy grids are often preferred when maps originate from SLAM or semantic segmentation pipelines due to their simplicity and direct compatibility with sensor data. Navigation meshes provide an efficient representation for large open areas with complex obstacle shapes, enabling smooth paths through convex regions while naturally handling varying terrain costs. Voronoi-based graphs are advantageous when vector maps with polygon boundaries are available and maximum clearance from obstacles is prioritized over path length. Visibility graphs yield the shortest Euclidean paths but may require additional safety margins since trajectories can pass close to obstacle corners.

Search Algorithm. We employ the A* algorithm [34] as the global planner to compute the shortest collision-free path between the start and the goal on the constructed graph. A* combines the exact path cost from the start with a heuristic estimate to the goal, and, given an admissible and consistent heuristic (e.g., Euclidean distance), it is both complete and optimal, typically expanding significantly fewer nodes than Dijkstra's algorithm [35]. In contrast, Breadth-First Search (BFS) [36] is optimal only on unweighted graphs

Algorithm 1 A* Algorithm

Require: occupancy map, start node s, goal node g, neighbor function $\mathcal{N}(\cdot)$, edge cost $c(\cdot,\cdot)$, heuristic $h(\cdot)$
Ensure: shortest path from s to g, if it exists

```
openSet ← {s}, closedSet ← ∅
cameFrom ← empty map
gScore(s) ← 0, fScore(s) ← h(s)
while openSet is not empty do
    n ← arg min_{x∈openSet} fScore(x)
    if n = g then
        return RECONSTRUCTPATH(cameFrom, n)
    end if
    remove n from openSet; add n to closedSet
    for each neighbor m ∈ N(n) do
        if m in closedSet or grid cell m is occupied then
            continue
        end if
        tentative ← gScore(n) + c(n, m)
        if m not in openSet then
            add m to openSet
        else if tentative ≥ gScore(m) then
            continue
        end if
        cameFrom(m) ← n
        gScore(m) ← tentative
        fScore(m) ← gScore(m) + h(m)
    end for
end while
return failure
function RECONSTRUCTPATH(cameFrom, n)
    path ← [n]
    while cameFrom contains n do
        n ← cameFrom(n)
        prepend n to path
    end while
    return path
end function
```

所有四种图构建方法都适用于我们的 HMP-DRL 框架；选择取决于可用的地图表示和所需的路径特征。当地图源自 SLAM 或语义分割管道时，占用网格通常是首选，因为它们简单且与传感器数据直接兼容。导航网格为具有复杂障碍物形状的大型开放区域提供了有效的表示，实现了穿过凸区域的平滑路径，同时自然地处理不同的地形成本。当具有多边形边界的矢量图可用并且与障碍物的最大间隙优先于路径长度时，基于 Voronoi 的图是有利的。可见性图产生最短的欧几里德路径，但可能需要额外的安全裕度，因为轨迹可以靠近障碍物角落。

搜索算法。我们采用 A* 算法 [34] 作为全局规划器来计算构造图上起点和目标之间的最短无碰撞路径。 A* 将从一开始的精确路径成本与对目标的启发式估计相结合，并且在给定可接受且一致的启发式（例如欧几里德距离）的情况下，它既完整又最优，通常比 Dijkstra 算法扩展的节点少得多 [35]。相反，广度优先搜索（BFS）[36]仅在未加权图上是最佳的

and explores many irrelevant states; Dijkstra's algorithm [35] is optimal with non-negative costs but lacks heuristic guidance, making it less efficient; incremental planners like D* [37] efficiently update paths when costs change but add unnecessary implementation complexity and computational overhead not required in our static global scenario [38].

The operation of the A* algorithm can be described as follows. An open list is maintained, ordered by priority $f(n) = g(n) + h(n)$, where g is the path cost from the start to node n, and h heuristically estimates the cost from n to the goal. At each step, the node with the minimum f is expanded, neighbors are relaxed, and the process repeats until the goal is selected; the optimal path is then reconstructed using backpointers. On the grid, we use occupancy checks to avoid static obstacles and a distance heuristic that preserves optimality. In our implementation, we use an 8-connected grid where each cell's neighbor set $\mathcal{N}(n)$ includes all 8 adjacent cells (4 orthogonal and 4 diagonal neighbors), enabling diagonal movement paths. For completeness, we provide the detailed pseudocode of the A* procedure in Algorithm 1.

Experimental Setup. In our experiments, the robot operates outdoors in an urban street environment (e.g., delivery robots, security patrol robots, and cleaning robots). We use the occupancy grid method: the map covers an area of 200 m by 200 m and is discretized into cells of 10 cm by 10 cm. The occupancy grid is formed from street semantic layers: areas such as buildings, lawns, vacant lots, sports fields, and other non-traversable or restricted zones are marked as occupied (forbidden for movement), while traversable road sections and pedestrian paths are marked as free space.

Examples of paths constructed using the A* algorithm in our experiments are shown in Fig. 1.

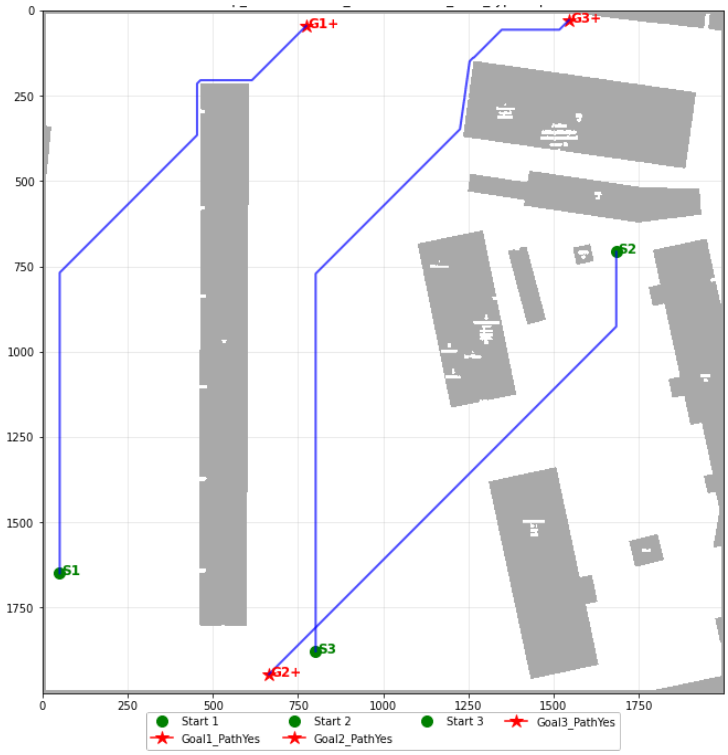

Figure 1: Three examples of global paths constructed by the A* algorithm for different start and goal pairs in our experiments. Occupied cells are marked in gray, which in this episode represent residential and non-residential buildings of various sizes and shapes in the city.

并探索许多不相关的状态；Dijkstra 算法[35]在非负成本下是最优的，但缺乏启发式指导，使其效率较低；像 D* [37] 这样的增量规划器在成本变化时可以有效地更新路径，但会增加不必要的实现复杂性和静态全局场景中不需要的计算开销 [38]。

A*算法的操作可以描述如下。维护一个开放列表，按优先级 $f(n) = g(n) + h(n)$ 排序，其中 g 是从起始点到节点 n 的路径成本，h 启发式估计从 n 到目标的成本。在每一步中，具有最小 f的节点被扩展，邻居被放松，并且重复该过程直到选择目标；然后使用反向指针重建最佳路径。在网格上，我们使用占用检查来避免静态障碍物，并使用距离启发式来保持最优性。在我们的实现中，我们使用 8 个连接的网格，其中每个单元的邻居集 $\mathcal{N}(n)$ 包括所有 8 个相邻单元（4 个正交邻居和 4 个对角邻居），从而实现对角移动路径。为了完整起见，我们提供了算法 1 中 A* 过程的详细伪代码。

实验设置。在我们的实验中，机器人在城市街道环境中的户外运行（例如送货机器人、安全巡逻机器人和清洁机器人）。我们使用占用网格方法：地图覆盖 200 m x 200 m 的面积，并离散化为 10 cm x 10 cm 的单元。占用网格由街道语义层组成：建筑物、草坪、空地、运动场和其他不可通行或限制区域等区域被标记为占用（禁止移动），而可通行路段和人行道被标记为自由空间。

我们实验中使用 A* 算法构建的路径示例如图 1 所示。

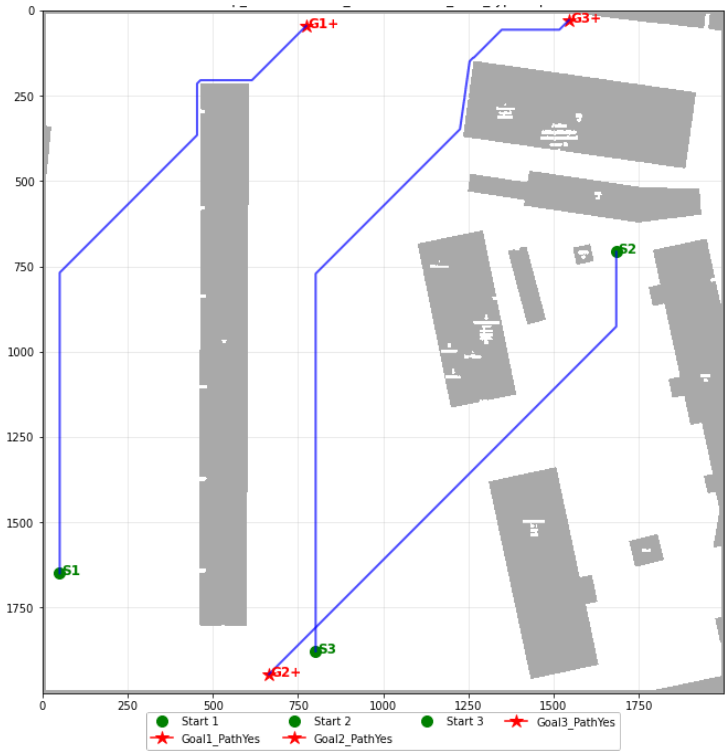

图 1：在我们的实验中，A* 算法针对不同的起始和目标对构建全局路径的三个示例。被占用的牢房被标记为灰色，在这一集中代表了城市中各种大小和形状的住宅和非住宅建筑。

4.2 Local Planning

Our local planning is based on the EB-CADRL approach described in [9], but with enhancements to incorporate the global path into the model and the reward function. The local planner considers both static and dynamic obstacles and aims to follow the global path.

To link global planning with local decision-making, we place a set of checkpoints along the global path. The checkpoints are positioned at a distance D from each other. The first checkpoint is located on the global path at a distance D from the start. Checkpoints can have different shapes and sizes. In our experiments, we use circular points with a fixed radius. These checkpoints are visible to the local planner and are included in both the reward function and the neural network V, encouraging progress along the global path.

Fig. 2 depicts a snapshot of the simulator during an HMP-DRL simulation. In it, we can see how checkpoints (green dashed circles) are positioned along the global path.

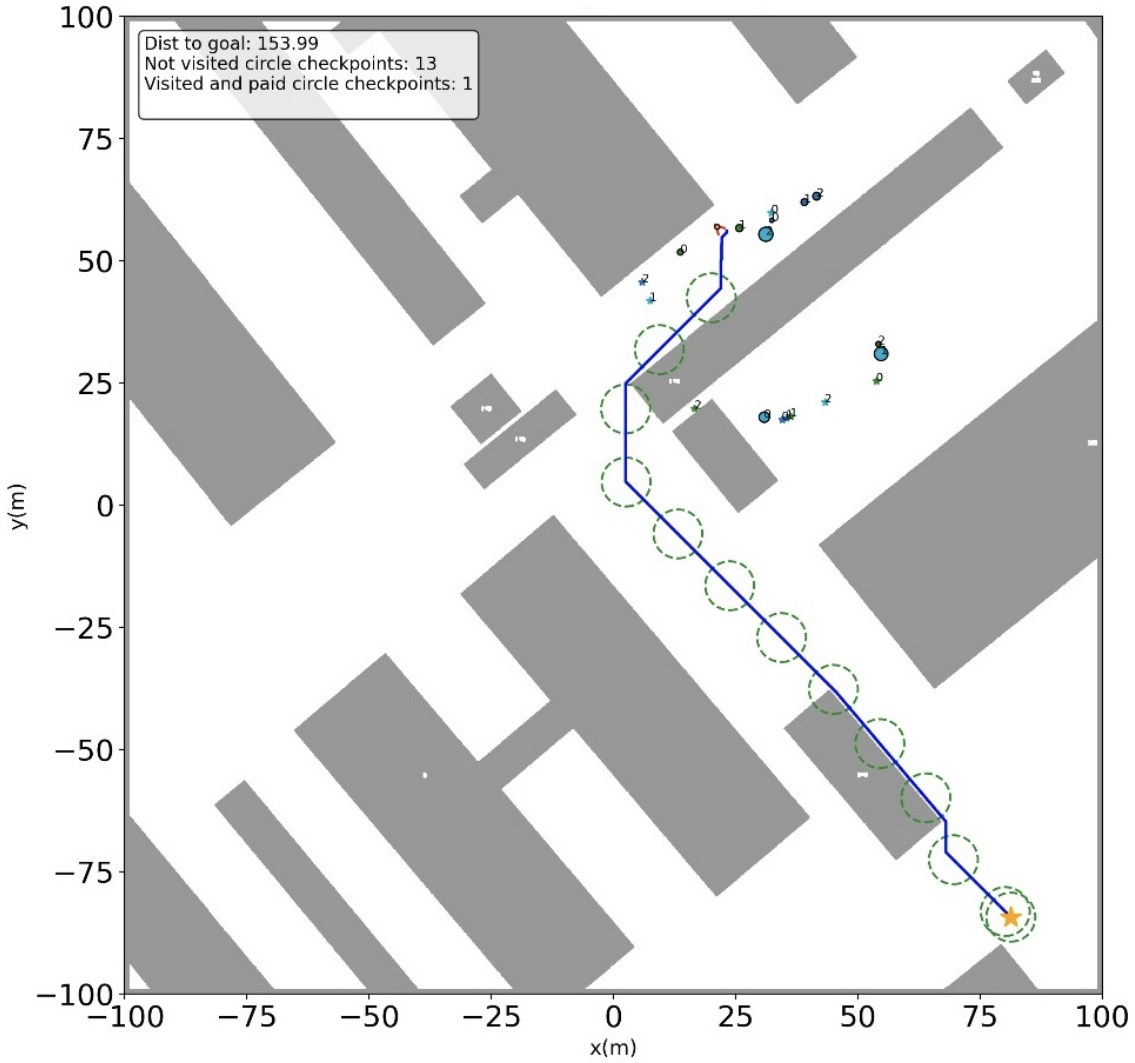

Figure 2: Simulator snapshot during the simulation of the HMP-DRL method. The blue polyline is the global path. Green circles are checkpoints. Gray objects are residential and non-residential buildings in the city. The remaining objects are the robot and the dynamic agents surrounding it.

4.2 地方规划

我们的本地规划基于 [9] 中描述的 EB-CADRL 方法，但进行了增强，将全局路径纳入模型和奖励函数中。局部规划者考虑静态和动态障碍，并旨在遵循全局路径。

为了将全球规划与本地决策联系起来，我们在全球路径上放置了一组检查点。检查点彼此之间的距离为 D。第一个检查点位于全局路径上，距起点的距离为 D。检查点可以有不同的形状和大小。在我们的实验中，我们使用具有固定半径的圆形点。这些检查点对本地规划者是可见的，并且包含在奖励函数和神经网络 V 中，鼓励沿着全局路径取得进展。

图 2 描绘了 HMP-DRL 模拟期间模拟器的快照。在其中，我们可以看到检查点（绿色虚线圆圈）如何沿着全局路径定位。

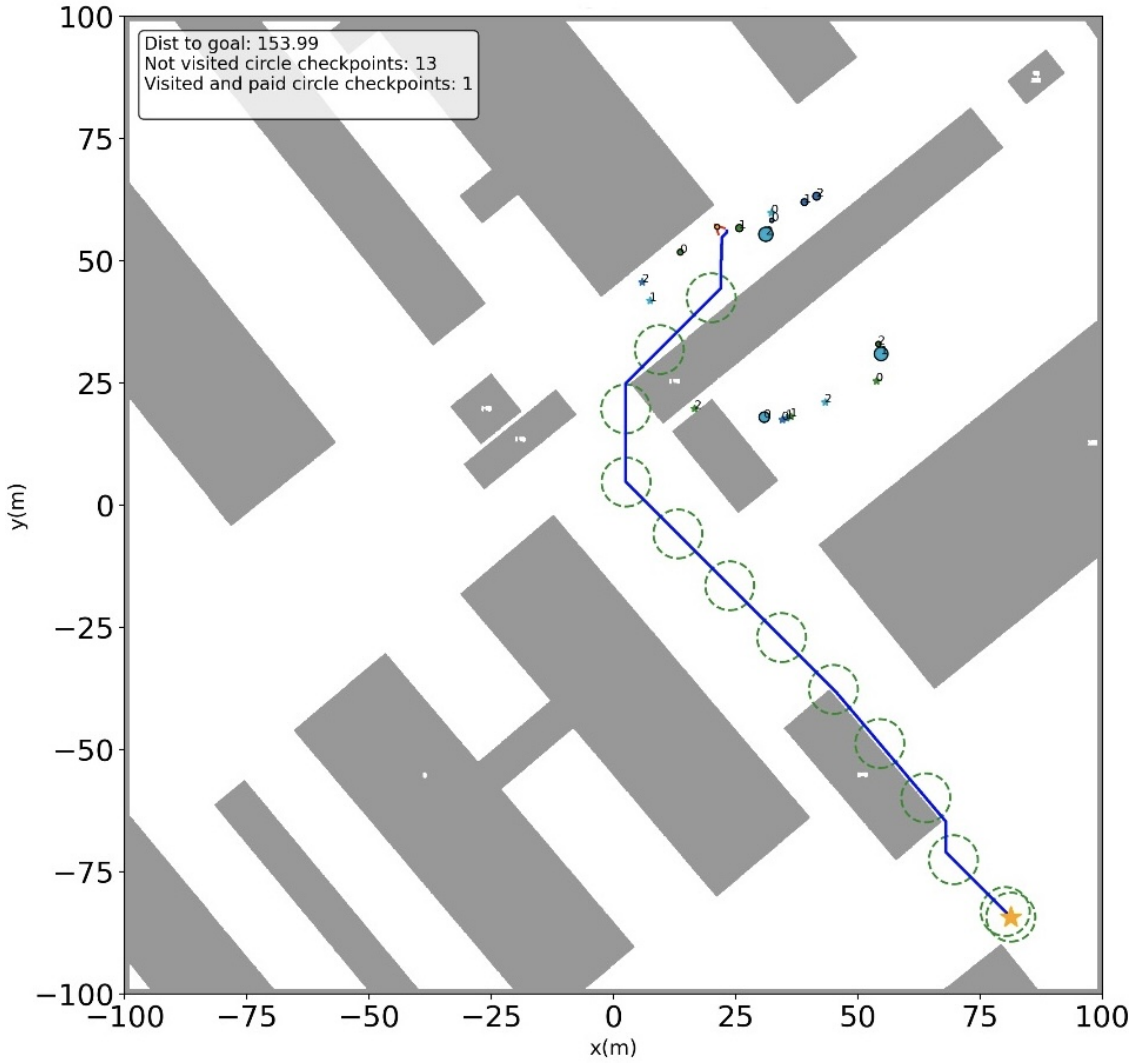

图 2：HMP-DRL 方法模拟过程中的模拟器快照。蓝色折线是全局路径。绿色圆圈是检查点。灰色物体是城市中的住宅和非住宅建筑。剩下的物体是机器人和它周围的动态代理。

4.3 Reward Function

Our reward function $R(t)$ comprises three additive components: a terminal reward $R_{\text{terminal}}(t)$ for episode-ending events (goal reached, collision, or timeout), a checkpoint reward $R_{\text{checkpoint}}(t)$ for progress along the global path, and a discomfort penalty $R_{\text{discomfort}}(t)$ for proximity violations.

Before defining these components, we establish notation for measuring robot-entity distances. Consider an environment with N entity types $\mathcal{E} = \{e_1, \ldots, e_N\}$. Let $d(e_i, t)$ denote the minimum distance between the robot and any entity of type e_i over the interval $[t - \Delta t, t]$. We define:

$$i^*(t) = \arg \min_{i \in \{1,\ldots,N\}} d(e_i, t), \tag{6}$$

where $i^*(t)$ is the index of the entity type closest to the robot. For brevity, we write i^* instead of $i^*(t)$. The minimum distance to any entity is $d_{\text{min}}(t) = d(e_{i^*}, t)$, and $e_{i^*} \in \mathcal{E}$ is the corresponding entity type. A collision occurs when $d_{\text{min}}(t) \leq 0$.

The robot receives a checkpoint reward $R_{\text{checkpoint}}(t)$ when it is inside a previously unvisited checkpoint along the A* path. Checkpoints may have different shapes and sizes. For example, we could use circular checkpoints, and the robot receives a reward when the Euclidean distance from the robot to the checkpoint's center is less than or equal to the checkpoint radius. Each checkpoint is paid out at most once (no repeated payments).

The function $R_{\text{time}}(t)$ represents the time reward:

$$R_{\text{time}}(t) = \begin{cases} 1 & \text{if } t < t_{\text{pref}} \\ \frac{t_{\text{max}} - t}{t_{\text{max}} - t_{\text{pref}}} & \text{if } t_{\text{pref}} \leq t \leq t_{\text{max}} \\ 0 & \text{otherwise} \end{cases} \tag{7}$$

where t_{pref} is the preferred time duration for reaching the goal, t_{max} is the maximum allowable time to reach the goal.

Let d_{max} be the maximum distance to the goal, which is the Euclidean distance between the starting position of the robot at time $t = 0$ and the position of the goal. Let $d_g(t)$ be the distance to the goal at time t. Then proximity reward can be represented as:

$$R_{\text{prox}}(t) = 1 - \frac{d_g(t)}{d_{\text{max}}}. \tag{8}$$

Define $R_{\text{coll}}(e_i)$ as the penalty associated with collisions involving an entity of type e_i, and R_{success} as the reward for reaching the goal.

Let $d_{\text{disc}}(e_i)$ be the discomfort distance for entity of type e_i and $p_{\text{disc}}(e_i)$ be the discomfort penalty factor for entity of type e_i, $i = 1, \ldots, N$.

Our reward function is defined as a sum of three components:

$$R(t) = R_{\text{terminal}}(t) + R_{\text{checkpoint}}(t) + R_{\text{discomfort}}(t). \tag{9}$$

The terminal reward handles mutually exclusive end-of-episode events and incorporates the proximity reward for timeout and collision cases:

$$R_{\text{terminal}}(t) = \begin{cases} R_{\text{prox}}(t) & \text{if } t \geq t_{\text{max}} \text{ and } \|\mathbf{p}_t - \mathbf{g}\|_2 > r \\ R_{\text{coll}}(e_{i^*}) + R_{\text{prox}}(t) & \text{else if } d_{\text{min}}(t) \leq 0 \\ R_{\text{success}} + R_{\text{time}}(t) & \text{else if } \|\mathbf{p}_t - \mathbf{g}\|_2 \leq r \\ 0 & \text{otherwise} \end{cases} \tag{10}$$

where r is the robot's radius. The proximity reward $R_{\text{prox}}(t)$ is added to timeout and collision cases to provide a learning signal proportional to the robot's progress toward the goal, even when the episode ends unsuccessfully.

4.3 奖励功能

我们的奖励函数 $R(t)$ 包含三个附加组件：针对情节结束事件（达到目标、碰撞或超时）的最终奖励 $R_{\text{terminal}}(t)$，针对全局路径进展的检查点奖励 $R_{\text{checkpoint}}(t)$，以及针对邻近违规的不适惩罚 $R_{\text{discomfort}}(t)$。

在定义这些组件之前，我们建立用于测量机器人实体距离的符号。考虑一个具有 N 实体类型 $\mathcal{E} = \{e_1, \ldots, e_N\}$ 的环境。令 $d(e_i, t)$ 表示机器人与任何 e_i 类型实体在区间 $[t - \Delta t, t]$ 上的最小距离。我们定义：

$$i^*(t) = \arg \min_{i \in \{1,\ldots,N\}} d(e_i, t), \tag{6}$$

其中$i^*(t)$是最接近机器人的实体类型的索引。为简洁起见，我们写为 i^* 而不是 $i^*(t)$。到任何实体的最小距离是$d_{\text{min}}(t) = d(e_{i^*}, t)$，$e_{i^*} \in \mathcal{E}$是相应的实体类型。当 $d_{\text{min}}(t) \leq 0$ 时发生冲突。

当机器人位于 A* 路径上先前未访问过的检查点内时，机器人会收到检查点奖励 $R_{\text{checkpoint}}(t)$。检查点可能有不同的形状和大小。例如，我们可以使用圆形检查点，当机器人到检查点中心的欧几里德距离小于或等于检查点半径时，机器人就会收到奖励。每个检查点最多支付一次（无重复支付）。

函数$R_{\text{time}}(t)$代表时间奖励：

$$R_{\text{time}}(t) = \begin{cases} 1 & \text{if } t < t_{\text{pref}} \\ \frac{t_{\text{max}} - t}{t_{\text{max}} - t_{\text{pref}}} & \text{if } t_{\text{pref}} \leq t \leq t_{\text{max}} \\ 0 & \text{otherwise} \end{cases} \tag{7}$$

其中 t_{pref} 是达到目标的首选持续时间，t_{max} 是达到目标的最大允许时间。

令 d_{max} 为到目标的最大距离，即机器人在时间 $t = 0$ 时的起始位置与目标位置之间的欧几里德距离。令$d_g(t)$ 为t 时刻到目标的距离。那么邻近奖励可以表示为：

$$R_{\text{prox}}(t) = 1 - \frac{d_g(t)}{d_{\text{max}}}. \tag{8}$$

将 $R_{\text{coll}}(e_i)$ 定义为与涉及 e_i 类型实体的碰撞相关的惩罚，将 R_{success} 定义为达到目标的奖励。

令 $d_{\text{disc}}(e_i)$ 为 e_i 类型实体的不适距离，$p_{\text{disc}}(e_i)$ 为 e_i、$i = 1, \ldots, N$ 类型实体的不适惩罚因子。

我们的奖励函数定义为三个组成部分的总和：

$$R(t) = R_{\text{terminal}}(t) + R_{\text{checkpoint}}(t) + R_{\text{discomfort}}(t). \tag{9}$$

终端奖励处理互斥的结束事件，并结合超时和碰撞情况的邻近奖励：

$$R_{\text{terminal}}(t) = \begin{cases} R_{\text{prox}}(t) & \text{if } t \geq t_{\text{max}} \text{ and } \|\mathbf{p}_t - \mathbf{g}\|_2 > r \\ R_{\text{coll}}(e_{i^*}) + R_{\text{prox}}(t) & \text{else if } d_{\text{min}}(t) \leq 0 \\ R_{\text{success}} + R_{\text{time}}(t) & \text{else if } \|\mathbf{p}_t - \mathbf{g}\|_2 \leq r \\ 0 & \text{otherwise} \end{cases} \tag{10}$$

其中 r 是机器人的半径。接近奖励 $R_{\text{prox}}(t)$ 被添加到超时和碰撞情况中，以提供与机器人实现目标的进度成比例的学习信号，即使情节不成功结束也是如此。

The checkpoint reward encourages progress along the global path:

$$R_{\text{checkpoint}}(t) = \begin{cases} R_{\text{cp}} & \text{if } \mathbf{p}_t \in \mathcal{C}_k \text{ and } k \text{ is unvisited} \\ 0 & \text{otherwise} \end{cases} \tag{11}$$

where $\mathcal{C}_k$ denotes the region of checkpoint k (e.g., a circle, rectangle, or line segment with a tolerance width), and R_{cp} is the checkpoint reward value. Each checkpoint yields a reward at most once.

The discomfort reward penalizes proximity to entities:

$$R_{\text{discomfort}}(t) = \begin{cases} (d(e_{i^*}, t) - d_{\text{disc}}(e_{i^*})) \cdot p_{\text{disc}}(e_{i^*}) \cdot \Delta t & \text{if } d(e_{i^*}, t) < d_{\text{disc}}(e_{i^*}) \text{ and } d_{\text{min}}(t) > 0 \\ 0 & \text{otherwise} \end{cases} \tag{12}$$

4.4 Model

Our model builds upon the architecture described in [9], extending it to incorporate information from the global path. The base architecture uses an attention mechanism to handle varying numbers of dynamic agents, while our extension integrates checkpoint features from the A* path to provide global guidance. We first describe the checkpoint feature extraction and extended self-state representation, then present the neural network architecture.

Checkpoint Feature Extraction. To incorporate global path information, we extract features from K upcoming checkpoints along the A* path. At each time step, we must determine which checkpoints to use as input features. Let m denote the index of the most recently visited checkpoint (with $m = -1$ initially). We consider two strategies for selecting the starting checkpoint index s. *Sequential selection*: The simplest approach sets $s = m + 1$, always selecting the K checkpoints immediately following the last visited one. This ensures strict progression along the path and provides consistent, deterministic features. *Nearest-first selection*: This adaptive approach searches among all unvisited checkpoints with indices $i > m$ and selects the one nearest to the robot's current position as the starting point: $s = \arg\min_{i>m} \|c_i - \mathbf{p}\|_2$. This strategy provides better local guidance when the robot deviates from the planned path, as it focuses on the most immediately relevant checkpoint rather than one that may be far ahead on the path.

The sequential strategy offers simplicity and stability, while the nearest-first strategy improves adaptability at a modest computational cost. In both cases, the constraint $s > m$ prevents the robot from receiving features for already-visited checkpoints, ensuring forward progress along the global path.

Let $\mathcal{C} = \{c_s, c_{s+1}, \ldots, c_{s+K-1}\}$ denote the set of K selected checkpoint positions. For each checkpoint $c_k = (c_x^k, c_y^k)$, we compute a feature vector in the robot-centric coordinate frame.

The robot-centric coordinate frame is defined such that the x-axis points from the robot's current position toward the goal. Let $\phi = \text{atan2}(g_y - p_y, g_x - p_x)$ be the rotation angle, where (p_x, p_y) is the robot position and (g_x, g_y) is the goal position. For each checkpoint c_k, we compute:

$$\begin{aligned} \delta_x^k &= c_x^k - p_x, \quad \delta_y^k = c_y^k - p_y, \\ \tilde{\delta}_x^k &= \delta_x^k \cos\phi + \delta_y^k \sin\phi, \\ \tilde{\delta}_y^k &= \delta_y^k \cos\phi - \delta_x^k \sin\phi, \\ d^k &= \sqrt{(\delta_x^k)^2 + (\delta_y^k)^2}, \end{aligned} \tag{13}$$

where $(\tilde{\delta}_x^k, \tilde{\delta}_y^k)$ are the rotated offsets in the robot-centric frame and d^k is the Euclidean distance to checkpoint k.

The checkpoint feature vector for each checkpoint is defined as:

$$\mathbf{f}^k = [d^k, \tilde{\delta}_x^k, \tilde{\delta}_y^k, r_{\text{cp}}], \tag{14}$$

where r_{cp} is the checkpoint radius. The complete checkpoint feature representation is the concatenation of all K checkpoint features:

$$\mathbf{f}_{\text{cp}} = [\mathbf{f}^1, \mathbf{f}^2, \ldots, \mathbf{f}^K]. \tag{15}$$

检查站奖励鼓励沿着全球路径取得进展：

$$R_{\text{checkpoint}}(t) = \begin{cases} R_{\text{cp}} & \text{if } \mathbf{p}_t \in \mathcal{C}_k \text{ and } k \text{ is unvisited} \\ 0 & \text{otherwise} \end{cases} \tag{11}$$

其中$\mathcal{C}_k$表示检查点k (的区域，例如具有公差宽度)的圆形、矩形或线段，R_{cp}是检查点奖励值。每个检查点最多产生一次奖励。
不适奖励会惩罚与实体的接近度：

$$R_{\text{discomfort}}(t) = \begin{cases} (d(e_{i^*}, t) - d_{\text{disc}}(e_{i^*})) \cdot p_{\text{disc}}(e_{i^*}) \cdot \Delta t & \text{if } d(e_{i^*}, t) < d_{\text{disc}}(e_{i^*}) \text{ and } d_{\text{min}}(t) > 0 \\ 0 & \text{otherwise} \end{cases} \tag{12}$$

4.4 模型

我们的模型建立在[9]中描述的架构之上，将其扩展以合并来自全局路径的信息。基本架构使用注意力机制来处理不同数量的动态代理，而我们的扩展集成了 A* 路径中的检查点功能以提供全局指导。我们首先描述检查点特征提取和扩展自状态表示，然后介绍神经网络架构。

检查点特征提取。为了合并全局路径信息，我们从 A* 路径上即将到来的 K 检查点中提取特征。在每个时间步，我们必须确定使用哪些检查点作为输入特征。令 m 表示最近访问的检查点的索引（最初为 $m = -1$）。我们考虑两种选择起始检查点索引 s 的策略。 *Sequential selection*：最简单的方法将 $s = m +$ 设置为 1，始终选择紧随最后访问的检查点之后的 K 检查点。这确保了沿着路径的严格进展并提供一致的、确定性的特征。 *Nearest-first selection*：这种自适应方法在所有未访问的索引为 $i > m$ 的检查点中进行搜索，并选择最接近机器人当前位置的一个作为起点：$s = \arg\min_{i>m} \|c_i - \mathbf{p}\|_2$。当机器人偏离计划路径时，该策略可以提供更好的本地指导，因为它专注于最直接相关的检查点，而不是路径上可能遥遥领先的检查点。
顺序策略提供了简单性和稳定性，而最近优先策略以适度的计算成本提高了适应性。在这两种情况下，约束 $s > m$ 都会阻止机器人接收已访问过的检查点的特征，从而确保沿着全局路径前进。

令 $\mathcal{C} = \{c_s, c_{s+1}, \ldots, c_{s+K-1}\}$ 表示 K 所选检查点位置的集合。对于每个检查点 $c_k = (c_x^k, c_y^k)$，我们计算以机器人为中心的坐标系中的特征向量。
定义以机器人为中心的坐标系，使得 x 轴从机器人的当前位置指向目标。令 $\phi = \text{atan2}(g_y - p_y, g_x - p_x)$ 为旋转角度，其中 (p_x, p_y) 为机器人位置，(g_x, g_y) 为目标位置。对于每个检查点 c_k，我们计算：

$$\begin{aligned} \delta_x^k &= c_x^k - p_x, \quad \delta_y^k = c_y^k - p_y, \\ \tilde{\delta}_x^k &= \delta_x^k \cos\phi + \delta_y^k \sin\phi, \\ \tilde{\delta}_y^k &= \delta_y^k \cos\phi - \delta_x^k \sin\phi, \\ d^k &= \sqrt{(\delta_x^k)^2 + (\delta_y^k)^2}, \end{aligned} \tag{13}$$

其中 $(\tilde{\delta}_x^k, \tilde{\delta}_y^k)$ 是以机器人为中心的框架中的旋转偏移量，d^k 是到检查点 k 的欧几里得距离。

这 定义每个检查点的检查点特征向量 作为：

$$\mathbf{f}^k = [d^k, \tilde{\delta}_x^k, \tilde{\delta}_y^k, r_{\text{cp}}], \tag{14}$$

其中 r_{cp} 是检查点半径。完整的检查点特征表示是所有 K 检查点特征的串联：

$$\mathbf{f}_{\text{cp}} = [\mathbf{f}^1, \mathbf{f}^2, \ldots, \mathbf{f}^K]. \tag{15}$$

Extended Self-State. In the base EB-CADRL model, the robot's self-state in the rotated frame is represented as $\mathbf{s}^r = [d_g, v_{\text{pref}}, \theta, r, v_x, v_y]$, where d_g is the distance to goal, v_{pref} is the preferred speed, θ is the heading angle, r is the robot radius, and (v_x, v_y) is the velocity in the robot-centric frame.

In our HMP-DRL model, we extend the self-state by incorporating the checkpoint features:

$$\tilde{\mathbf{s}}^r = [d_g, v_{\text{pref}}, \theta, r, v_x, v_y, \mathbf{f}_{\text{cp}}], \tag{16}$$

resulting in an extended self-state of dimension $6 + 4K$. This extended self-state provides the model with explicit information about the global path, enabling it to make decisions that balance local collision avoidance with progress toward distant goals.

Entity State Embedding. We embed the extended robot state $\tilde{\mathbf{s}}^r$, along with the observable state of each entity i, into a fixed-length vector g_i using a multi-layer perceptron (MLP):

$$g_i = \phi_g(\tilde{\mathbf{s}}^r, \mathbf{s}^{io}; W_g), \tag{17}$$

where $\phi_g(\cdot)$ is an embedding function with rectified linear unit (ReLU) activations, and W_g are the embedding weights. To account for different types of entities (e.g., adults, children, bicycles), we use one-hot encoding to represent each agent type. These encoded features are concatenated with the primary features of the agents and passed into the model.

The embedding vector g_i is then fed into another MLP to obtain the pairwise interaction feature between the robot and entity i:

$$h_i = \psi_h(g_i; W_h), \tag{18}$$

where $\psi_h(\cdot)$ is a fully-connected layer with ReLU nonlinearity, and W_h are the network weights.

Social Attentive Pooling. To handle varying numbers of entities, we use a social attentive pooling module [8] which learns the relative importance of each neighbor and the collective impact of the crowd in a data-driven manner. Let n denote the number of entities in the environment. The interaction embedding g_i is transformed into an attention score α_i as follows:

$$g_m = \frac{1}{n}\sum_{k=1}^{n} g_k, \qquad \alpha_i = \psi_\alpha(g_i, g_m; W_\alpha), \tag{19}$$

where g_m is a fixed-length embedding vector obtained by mean pooling all individuals, $\psi_\alpha(\cdot)$ is an MLP with ReLU activations, and W_α are the weights.

Given the pairwise interaction vector h_i and the corresponding attention score α_i for each neighbor i, the final representation of the crowd is a weighted linear combination of all pairs:

$$c = \sum_{i=1}^{n} \text{softmax}(\alpha_i) h_i, \tag{20}$$

where $\text{softmax}(\alpha_i) = \frac{e^{\alpha_i}}{\sum_{j=1}^{n} e^{\alpha_j}}$.

Value Network. Using the compact representation of the crowd c and the extended self-state $\tilde{\mathbf{s}}^r$, we construct a planning module that estimates the state value v for cooperative planning:

$$v = f_v(\tilde{\mathbf{s}}^r, c; W_v), \tag{21}$$

where $f_v(\cdot)$ is an MLP with ReLU activations, and W_v are the weights. We denote this value network as V.

In our model checkpoint features are incorporated into the extended self-state $\tilde{\mathbf{s}}^r$, which is then used throughout the entire network pipeline from the initial pairwise embedding ϕ_g through to the final value estimation f_v. This allows the network to learn representations that jointly consider both dynamic obstacle avoidance and global path following from the very first layer. During training, the network learns to balance following the global path (guided by checkpoint rewards) with avoiding dynamic obstacles (guided by collision and discomfort penalties).

扩展的自状态。在基本 EB-CADRL 模型中，机器人在旋转坐标系中的自状态表示为 $\mathbf{s}^r = [d_g, v_{\text{pref}}, \theta, r, v_x, v_y]$，其中 d_g 是到目标的距离，v_{pref} 是首选速度，θ 是航向角，r 是机器人半径，(v_x, v_y) 是以机器人为中心的坐标系中的速度。
在我们的 HMP-DRL 模型中，我们通过合并检查点功能来扩展自状态：

$$\tilde{\mathbf{s}}^r = [d_g, v_{\text{pref}}, \theta, r, v_x, v_y, \mathbf{f}_{\text{cp}}], \tag{16}$$

导致维度 $6 + 4K$ 的扩展自状态。这种扩展的自我状态为模型提供了有关全局路径的明确信息，使其能够做出平衡局部碰撞避免与实现遥远目标的进展的决策。

实体状态嵌入。我们使用多层感知器（MLP）将扩展机器人状态 $\tilde{\mathbf{s}}^r$ 以及每个实体 i 的可观察状态嵌入到固定长度向量 g_i 中：

$$g_i = \phi_g(\tilde{\mathbf{s}}^r, \mathbf{s}^{io}; W_g), \tag{17}$$

其中 $\phi_g(\cdot)$ 是具有修正线性单元 (ReLU) 激活的嵌入函数，W_g 是嵌入权重。为了考虑不同类型的实体（例如成人、儿童、自行车），我们使用 one-hot 编码来表示每种代理类型。这些编码特征与代理的主要特征连接并传递到模型中。

然后将嵌入向量 g_i 输入到另一个 MLP 中，以获得机器人和实体 i 之间的成对交互特征：

$$h_i = \psi_h(g_i; W_h), \tag{18}$$

其中$\psi_h(\cdot)$是具有ReLU非线性的全连接层，W_h是网络权重。

社会关注汇集。为了处理不同数量的实体，我们使用社交注意力池模块[8]，该模块以数据驱动的方式学习每个邻居的相对重要性以及人群的集体影响。让 n 表示环境中的实体数量。交互嵌入 g_i 转换为注意力分数 α_i，如下所示：

$$g_m = \frac{1}{n}\sum_{k=1}^{n} g_k, \qquad \alpha_i = \psi_\alpha(g_i, g_m; W_\alpha), \tag{19}$$

其中g_m是通过平均池化所有个体获得的固定长度嵌入向量，$\psi_\alpha(\cdot)$是具有ReLU激活的MLP，W_α是权重。
给定成对交互向量 h_i 和每个邻居 i 的相应注意力分数 α_i，人群的最终表示是所有对的加权线性组合：

$$c = \sum_{i=1}^{n} \text{softmax}(\alpha_i) h_i, \tag{20}$$

其中 $\text{softmax}(\alpha_i) = \frac{e^{\alpha_i}}{\sum_{j=1}^{n} e^{\alpha_j}}$.

价值网络。使用人群 c 的紧凑表示和扩展的自我状态 $\tilde{\mathbf{s}}^r$，我们构建了一个规划模块，用于估计合作规划的状态值 v：

$$v = f_v(\tilde{\mathbf{s}}^r, c; W_v), \tag{21}$$

其中 $f_v(\cdot)$ 是具有 ReLU 激活的 MLP，W_v 是权重。我们将此价值网络表示为 V。
在我们的模型中，检查点特征被合并到扩展的自状态 $\tilde{\mathbf{s}}^r$ 中，然后在整个网络管道中使用它，从初始的成对嵌入 ϕ_g 到最终的值估计 f_v。这使得网络能够从第一层学习联合考虑动态避障和全局路径跟踪的表示。在训练过程中，网络学习在遵循全局路径（由检查点奖励引导）和避免动态障碍（由碰撞和不适惩罚引导）之间取得平衡。

4.5 Algorithm HMP-DRL

The HMP-DRL algorithm integrates global path planning with local reinforcement learning through Parallel Deep V-learning. The training procedure runs multiple episodes in parallel, collects experience, and updates the value network V using temporal-difference learning with experience replay and a fixed target network $\hat{V}$. The main training loop is described in Algorithm 2, and the episode execution is described in Algorithm 3.

Algorithm 2 HMP-DRL Training with Parallel Deep V-learning

Run imitation learning with demonstration D, update value network V with D
Initialize: target value network $\hat{V} \leftarrow V$, experience replay memory $E \leftarrow D$, parallel processes number N
while episode $<$ M **do**
  Initialize a multiprocessing Pool of size N
  for k = episode, episode + N **do** in parallel:
    Execute episode k using Algorithm 3
  end for
  Sample random minibatch tuples from E
  Update value network V by gradient descent
  if episode $\mod$ UpdateInterval = 0 **then**
    Update target value network $\hat{V} \leftarrow V$
  end if
  Update $episode \leftarrow episode + N$
end while
return V

Algorithm 3 HMP-DRL Episode Execution

Initialize environment with random start $\mathbf{p}_0$ and goal $\mathbf{g}$
Compute global path $\mathcal{P} \leftarrow \text{A}^*(\text{OccupancyMap}, \mathbf{p}_0, \mathbf{g})$
Place checkpoints $\{c_1, \ldots, c_L\}$ along $\mathcal{P}$ at intervals D
Initialize: visited set $\mathcal{V} \leftarrow \emptyset$, buffer $S \leftarrow \emptyset$, $t \leftarrow 0$
repeat
  Set last visited checkpoint index: $m \leftarrow \max\{i : c_i \in \mathcal{V}\}$
  Select K checkpoints: $\mathcal{C}_K \leftarrow \{c_{m+1}, \ldots, c_{\min(m+K,L)}\}$
  Compute checkpoint features $\mathbf{f}_{\text{cp}}$ using (13)–(15)
  Construct extended state $\tilde{\mathbf{s}}_t^r \leftarrow [d_g, v_{\text{pref}}, \theta, r, v_x, v_y, \mathbf{f}_{\text{cp}}]$
  Form joint state $\tilde{\mathbf{s}}_t^j \leftarrow [\tilde{\mathbf{s}}_t^r, \mathbf{s}_t^{1o}, \ldots, \mathbf{s}_t^{no}]$
  $\mathbf{a}_t \leftarrow \begin{cases} \text{RandomAction}(), & \epsilon \\ \arg\max_{\mathbf{a}\in\mathbf{A}} R(\tilde{\mathbf{s}}_t^j, \mathbf{a}) + \gamma^{\Delta t \cdot v_{\text{pref}}} V(\tilde{\mathbf{s}}_{t+\Delta t}^j), & 1-\epsilon \end{cases}$
  Execute $\mathbf{a}_t$, observe r_t and next state
  if robot enters unvisited checkpoint c_i **then**
    $\mathcal{V} \leftarrow \mathcal{V} \cup \{c_i\}$, add R_{cp} to r_t
  end if
  Store $(\tilde{\mathbf{s}}_t^j, \mathbf{a}_t, r_t, \tilde{\mathbf{s}}_{t+\Delta t}^j)$ in S
  $t \leftarrow t + \Delta t$
until terminal state or $t \geq t_{\max}$
for each $(\tilde{\mathbf{s}}_t^j, \mathbf{a}_t, r_t, \tilde{\mathbf{s}}_{t+\Delta t}^j)$ in S **do**
  $\text{value}_t \leftarrow r_t + \gamma^{\Delta t \cdot v_{\text{pref}}} \hat{V}(\tilde{\mathbf{s}}_{t+\Delta t}^j)$
  Store $(\tilde{\mathbf{s}}_t^j, \text{value}_t)$ in experience memory E
end for

At the beginning of each episode, we compute the global path $\mathcal{P}$ from the robot's start position to its goal using A* on the occupancy grid. We then place checkpoints $\{c_1, \ldots, c_L\}$ along $\mathcal{P}$ at intervals of distance D.

4.5 算法HMP-DRL

HMP-DRL算法通过并行深度V学习将全局路径规划与局部强化学习相结合。训练过程并行运行多个episode，收集经验，并使用带有经验回放的时差学习和固定目标网络$\hat{V}$来更新价值网络 V。主训练循环如算法 2 所示，情节执行如算法 3 所示。

算法 2 采用并行深度 V 学习的 HMP-DRL 训练

1: 通过演示 D 运行模仿学习，用 D 更新价值网络 $V$2: 初始化：目标价值网络$\hat{V} \leftarrow V$，体验回放记忆 $E \leftarrow D$，并行进程数 $N$3: while Episode $<$ M do 4: 初始化大小为 N 的多处理池 5: 对于 k = Episode，episode + N并行执行： 6: 执行 Episode k 使用算法 3 7: end for 8: 从 E 中采样随机小批量元组 9: 通过梯度下降更新值网络 $V$10: if Episode mod UpdateInterval = 0 then 11: 更新目标值网络$\hat{V} \leftarrow V$12: end if 13: 更新 $episode \leftarrow episode + N$14: end while 15: 返回 V

算法 3 HMP-DRL 片段执行

1: 初始化环境，随机开始 $\mathbf{p}0$ 和目标 $\mathbf{g}$ 2: 计算全局路径 $\mathcal{P} \leftarrow$ A*(OccupancyMap, $\mathbf{p}0, \mathbf{g}$) 3: 沿 $\mathcal{P}$ 每隔 D 放置检查点 $\{c_1, \ldots, c_L\}$ 4: 初始化：访问集 $\mathcal{V} \leftarrow \emptyset$、缓冲区 $S \leftarrow \emptyset$、$t \leftarrow 0$ 5:重复 6: 设置上次访问的检查点索引: $m \leftarrow \max\{i : c_i \in \mathcal{V}\}$ 7: 选择 K 检查点: $\mathcal{C}_K \leftarrow \{c_{m+1}, \ldots, c_{\min(m+K,L)}\}$ 8: 使用 (13) – (15) 计算检查点特征 $\mathbf{f}_{\text{cp}}$ 9: 构造扩展状态 $\tilde{\mathbf{s}}_t^r \leftarrow [d_g, v_{\text{pref}}, \theta, r, v_x, v_y, \mathbf{f}_{\text{cp}}]$ 10: 形成联合状态 $\tilde{\mathbf{s}}_t^j \leftarrow [\tilde{\mathbf{s}}_t^r, \mathbf{s}_t^1, \ldots, \mathbf{s}_t^{no}]$ 11: $\mathbf{a}_t \leftarrow \begin{cases} \text{RandomAction()} & \epsilon \\ \arg\max_{\mathbf{a} \in \mathbf{A}} R(\tilde{\mathbf{s}}_t^j, \mathbf{a}) + \gamma^{\Delta t \cdot v_{\text{pref}}} V(\tilde{\mathbf{s}}_{t+\Delta t}^j), & 1-\epsilon \end{cases}$ 12: 执行 $\mathbf{a}_t$，观察 r_t 和下一个状态 13: 如果机器人进入未访问的检查点 c_i 则 14: $\mathcal{V} \leftarrow \mathcal{V} \cup \{c_i\}$，添加$R_{\text{cp}}$ 到 r_t 15: 结束 if 16: 将 ($\tilde{\mathbf{s}}_t^j, \mathbf{a}_t, r_t, \tilde{\mathbf{s}}_{t+\Delta t}^j$) 存储在 S 17: $t \leftarrow t + \Delta t$ 18: 直到终端状态或 $t \geq t_{\max}$ 19: 对于 S 中的每个 ($\tilde{\mathbf{s}}_t^j, \mathbf{a}_t, r_t, \tilde{\mathbf{s}}_{t+\Delta t}^j$) 执行 20: $\text{value}_t \leftarrow r_t + \gamma^{\Delta t \cdot v_{\text{pref}}} \hat{V}(\tilde{\mathbf{s}}_{t+\Delta t}^j)$ 21: 将 ($\tilde{\mathbf{s}}_t^j$, value_t) 存储在经验记忆 E 中22：结束

At the beginning of each episode, we compute the global path $\mathcal{P}$ from the robot's start position to its goal using A* on the occupancy grid. We then place checkpoints $\{c_1, \ldots, c_L\}$ along $\mathcal{P}$ at intervals of distance D.

At each time step, we select K upcoming checkpoints, transform them into the robot-centric frame using (13), and concatenate the resulting features $\mathbf{f}_{\text{cp}}$ with the base self-state to form $\tilde{\mathbf{s}}^r$ as defined in (16). The extended joint state $\tilde{\mathbf{s}}_t^j = [\tilde{\mathbf{s}}_t^r, \mathbf{s}_t^{1o}, \ldots, \mathbf{s}_t^{no}]$ is then used for action selection and value estimation.

The global path is computed once per episode, avoiding the computational overhead of replanning at every step. Since each episode has a unique start-goal pair, the robot learns to follow diverse paths through the environment. The checkpoint features provide the local planner with global context, enabling navigation over long distances while the attention-based value network handles dynamic obstacle avoidance.

4.6 Implementation Details

We assume non-holonomic robot kinematics and use a unicycle model. The action space consists of 81 discrete actions: a stop action (zero velocity) plus combinations of 5 speeds, exponentially distributed in the range (0, v_{pref}], and 16 directions, uniformly distributed in $[0, 2\pi)$.

For the global path representation, we use $K = 2$ checkpoints with radius $r_{\text{cp}} = 5\,\text{m}$ and inter-checkpoint distance $D = 15$. Note that K denotes the number of checkpoints fed to the model at each timestep, not the total number of checkpoints along the path, which depends on the path length and distance D. This results in $4K = 8$ checkpoint features appended to the base self-state, yielding an extended self-state dimension of $6 + 8 = 14$.

The neural network architecture consists of four multi-layer perceptrons (MLP). An MLP's structure is denoted by the tuple $(d_{\text{in}}, d_1, \ldots, d_{\text{out}})$, which specifies the number of neurons in the input, hidden, and output layers, respectively. The dimensions of these MLP components are ϕ_g: (25, 300, 200), ψ_h: (200, 200, 100), ψ_α: (400, 200, 200, 1), and f_v: (114, 350, 250, 200, 1). The input dimension 25 to ϕ_g represents the joint state consisting of the extended robot state (14), agent's observable state (7), and entity type one-hot encoding (4). The 114 input dimensions to f_v combine the extended self-state (14) with the attention-weighted feature representation of all agents (100).

We apply batch normalization [39] after each hidden linear layer and before the ReLU activation in all MLPs. This technique standardizes the inputs to each layer, which stabilizes training and allows for the use of higher learning rates. Additionally, it accelerates convergence and provides a mild regularization effect that can help mitigate overfitting. In our experiments, batch normalization proved particularly beneficial for stability in the value-based reinforcement learning setting, where target values are non-stationary.

We implemented the policy in the deep learning framework PyTorch [40] and trained it with a batch size of 100 (for each episode) using stochastic gradient descent. For imitation learning, we collected 3000 episodes of demonstration using ORCA and trained the policy 200 epochs with a learning rate of 0.01. For reinforcement learning, the learning rate is 0.001 and the discount factor γ is 0.99. The exploration rate of the ϵ-greedy policy decays linearly from 0.5 to 0.05 in the first 25000 episodes and stays at 0.05 for the remaining episodes.

In our experiments, all input features (robot state, entity states, and entity types) are obtained directly from the simulator. On a real robot, these would be provided by standard localization and perception systems [41], whose implementation is beyond the scope of this work.

5 Experiments

5.1 Urban Episode Generation

We generate a large dataset of robot navigation episodes in a simulator, based on real urban geometry from a map, by rasterizing OpenStreetMap (OSM) data for a large European city [42]. The data preparation pipeline generates map snapshots, semantically segments each snapshot into land use and infrastructure categories, collapses the semantics into a binary occupancy map for global planning, and finally selects start-goal pairs between which a path is guaranteed to exist. These maps are subsequently used as static layers, while dynamic agents are modeled during training and evaluation using ORCA [11] under the "invisible robot" assumption.

在每个时间步，我们选择 K 即将到来的检查点，使用（13）将它们转换为以机器人为中心的框架，并将结果特征 $\mathbf{f}_{\text{cp}}$ 与基本自我状态连接起来形成 $\tilde{\mathbf{s}}^r$ （如（16）中定义的）。然后，扩展的关节状态 $\tilde{\mathbf{s}}_t^j = [\tilde{\mathbf{s}}_t^r, \mathbf{s}_t^{1o}, \ldots, \mathbf{s}_t^{no}]$ 用于动作选择和价值估计。
全局路径每集计算一次，避免了每一步重新规划的计算开销。由于每一集都有一个独特的起始目标对，因此机器人学会在环境中遵循不同的路径。检查点功能为本地规划者提供全局背景，实现长距离导航，同时基于注意力的价值网络处理动态避障。

4.6 实施细节

我们假设非完整机器人运动学并使用独轮车模型。动作空间由 81 个离散动作组成：一个停止动作（零速度）加上 5 个速度的组合，在范围 $(0, v_{\text{pref}}]$ 内呈指数分布，以及 16 个方向，在 $[0, 2\pi)$ 内均匀分布。
对于全局路径表示，我们使用 $K = 2$ 个检查点，半径 $r_{\text{cp}} = 5$ m，检查点间距离 $D = 15$。请注意，K 表示在每个时间步馈送到模型的检查点数量，而不是沿路径的检查点总数，这取决于路径长度和距离 D。这会导致 $4K = 8$ 个检查点特征附加到基本自状态，从而产生 $6 + 8 = 14$ 的扩展自状态维度。

神经网络架构由四个多层感知器（MLP）组成。 MLP 的结构由元组 $(d_{\text{in}}, d_1, \ldots, d_{\text{out}})$ 表示，它分别指定输入层、隐藏层和输出层中的神经元数量。这些 MLP 组件的维度为 ϕ_g: (25, 300, 200)、ψ_h: (200, 200, 100)、ψ_α: (400, 200, 200, 1) 和 f_v: (114, 350, 250, 200, 1)。输入维度 25 到 ϕ_g 表示由扩展机器人状态 (14)、代理可观察状态 (7) 和实体类型 one-hot 编码 (4) 组成的关节状态。 f_v 的 114 个输入维度将扩展的自我状态 (14) 与所有代理的注意力加权特征表示 (100) 结合起来。

我们在所有 MLP 中的每个隐藏线性层之后和 ReLU 激活之前应用批量归一化 [39]。该技术标准化了每一层的输入，从而稳定了训练并允许使用更高的学习率。此外，它还可以加速收敛并提供温和的正则化效果，有助于减轻过度拟合。在我们的实验中，事实证明，批量归一化对于基于值的强化学习设置（其中目标值是非平稳的）的稳定性特别有益。

我们在深度学习框架 PyTorch [40] 中实现了该策略，并使用随机梯度下降以 100 的批量大小（每个情节）对其进行训练。对于模仿学习，我们使用 ORCA 收集了 3000 集演示，并以 0.01 的学习率对策略进行了 200 轮训练。对于强化学习，学习率为 0.001，折扣因子 γ 为 0.99。 ϵ 贪婪策略的探索率在前 25000 个回合中从 0.5 线性衰减到 0.05，并在其余回合中保持在 0.05。

在我们的实验中，所有输入特征（机器人状态、实体状态和实体类型）都是直接从模拟器获得的。在真正的机器人上，这些将由标准定位和感知系统提供[41]，其实现超出了本工作的范围。

5 实验

5.1 城市情节生成

我们根据地图中的真实城市几何形状，通过栅格化欧洲大城市的 OpenStreetMap (OSM) 数据，在模拟器中生成机器人导航事件的大型数据集 [42]。数据准备管道生成地图快照，在语义上将每个快照分割为土地利用和基础设施类别，将语义折叠成用于全局规划的二进制占用地图，最后选择保证之间存在路径的起始目标对。这些地图随后被用作静态层，而动态代理在 “ 隐形机器人 ” 假设下使用 ORCA [11] 进行训练和评估期间建模。

Map Snapshots from OSM. We define a rectangular area centered on the center of the city and uniformly discretize it with a dense grid of viewpoints within a 29 km $\times$ 38 km region. For each viewpoint, we generate a high-zoom square raster snapshot (zoom 20) by stitching 256 px map tiles from a self-hosted OSM tile server. Snapshots are rendered without overlay text labels to ensure a stable color palette and are saved as square images (2000 px per side). This step follows the standard Web Mercator projection tiling procedure: geographic coordinates are converted to tile coordinates at the target zoom level, neighboring tiles covering the desired area are retrieved, and the resulting mosaic is cropped to the required size.

Each snapshot is converted into a semantic raster by matching pixel colors to a fixed palette. We use the nearest color rule (Manhattan distance in RGB with a small threshold) to assign each image pixel to one of the following classes:

1. Residential area
2. Greenfield land
3. Industrial area
4. School/University/Hospital
5. Allotments
6. Parking
7. Brownfield land
8. Park / green zone
9. Forest / park
10. Road
11. Commercial area
12. Grass / meadow
13. Sports pitch
14. Water
15. Building

Binary Occupancy Map. Next, the semantic raster is collapsed into a binary occupancy map encoding static traversability for the global planner. Specifically, classes corresponding to natural obstacles or impassable infrastructure (buildings, water, sports pitches, and grass/meadow) are marked as occupied, while all other classes (residential areas with sidewalk, soil with sidewalk, industrial areas, schools/universities/hospitals, allotments, parking, brownfield sites, parks/green zones, forests/parks, roads, and commercial areas) are marked as free.

In Figs. 3a and 3b, we can see an example of the steps described above: an occupancy map is generated from an OSM city map, start and finish points are placed, and a global path is constructed.

(a) OSM map snapshot. (b) Binary occupancy map and global path.

Figure 3: Example of map processing: (a) an OSM map snapshot from which occupancy maps were formed; (b) the corresponding binary occupancy map with a global path constructed using A*.

来自 OSM 的地图快照。我们定义了一个以城市中心为中心的矩形区域，并在 29 公里 × 38 公里区域内使用密集的视点网格对其进行统一离散化。对于每个视点，我们通过拼接来自自托管 OSM 切片服务器的 256 像素地图切片来生成高缩放方形栅格快照（缩放 20）。快照在渲染时没有叠加文本标签，以确保稳定的调色板，并保存为方形图像（每边 2000 像素）。此步骤遵循标准的 Web 墨卡托投影切片程序：将地理坐标转换为目标缩放级别的切片坐标，检索覆盖所需区域的相邻切片，并将生成的马赛克裁剪为所需的大小。
通过将像素颜色与固定调色板匹配，将每个快照转换为语义栅格。我们使用最近的颜色规则（RGB 中的曼哈顿距离，阈值较小）将每个图像像素分配给以下类别之一：

1. 住宅区 2. 绿地 3. 工业区 4. 学校/大学/医院 5. 土地 6. 停车场 7. 棕地 8. 公园/绿地 9. 森林/公园 10. 道路 11. 商业区 12. 草地/草地 13. 运动场 14. 水域 15. 建筑

二进制占用图。接下来，语义栅格被折叠成二进制占用图，为全局规划器编码静态可遍历性。具体来说，与自然障碍或无法通行的基础设施（建筑物、水、运动场和草地/草地）相对应的类别被标记为占用，而所有其他类别（有人行道的住宅区、有人行道的土壤、工业区、学校/大学/医院、分配地、停车场、棕地、公园/绿地、森林/公园、道路和商业区）被标记为空闲。

在图中。在图3a和3b中，我们可以看到上述步骤的示例：从OSM城市地图生成占用地图，放置起点和终点，并构建全局路径。

(a) OSM map snapshot. (b) Binary occupancy map and global path.

图 3：地图处理示例：(a) OSM 地图快照，从中形成占用地图； (b) 相应的二进制占用图以及使用 A* 构建的全局路径。

Start-Goal Sampling. On each binary map, we select between one and three start-goal pairs. The robot's footprint is modeled as a 20×20 pixel square; valid start and goal positions are those where the entire footprint lies completely within free space. Additionally, we require the goal to be sufficiently far from the start to yield non-trivial episodes: we select goals at a Euclidean distance of at least 0.75 of the map's larger dimension. To ensure dynamic feasibility at the global level, we inflate obstacles by half the footprint thickness and perform an 8-connected A* search (as described in Section 4) on the inflated grid. Pairs are accepted only if a collision-free A* path exists; accepted paths are saved along with the start and goal.

Resulting Dataset. Since we aim to train the robot to drive in sufficiently complex environments, we discard maps with no or very few static obstacles by introducing a minimum static occupancy fraction: maps with an obstacle percentage below the threshold (5%) are skipped and unused. Maps for which no valid start-goal pair satisfies the path existence constraint are also removed.

After filtering and removing unrealizable cases, we obtain 30,948 distinct maps. Sampling 1–3 start-goal pairs per map with a guaranteed path from start to finish yields 77,663 unique navigation episodes. These episodes serve as static occupancy maps within our simulator. We randomly split them into training, validation, and testing sets: for our main experiments, we use 60,000 episodes for training, 500 for validation, and 1,000 for testing. The remaining episodes are reserved for additional experiments as needed.

5.2 Reward setup

In this subsection, we define the reward function for our experiments. The robot navigates in an environment with four entity types: adults, bicycles, children, and static obstacles. Each dynamic entity type has 4 instances, resulting in 12 dynamic agents per scenario. Static obstacles are defined by an occupancy map and vary by region.

$$R_{\text{checkpoint}}(t) = \begin{cases} 0.3, & \text{if } \|\mathbf{p}_t - \mathbf{c}_k\|_2 < r_{\text{checkpoint}} \text{ and } k \text{ is unvisited checkpoint} \\ 0, & \text{otherwise} \end{cases} \tag{22}$$

$$R_{\text{success}} = 3 \tag{23}$$

We denote the entity types as follows: Adult (A), Bicycle (B), Child (C), and Obstacle (O). The penalties for collisions are defined as:

$$R_{\text{coll}}(e) = \begin{cases} -1.0, & \text{if } e = O \\ -1.5, & \text{if } e = A \\ -2.0, & \text{if } e = B \\ -2.5, & \text{if } e = C \end{cases} \tag{24}$$

We scale collision penalties according to unpredictability and harm potential. Static obstacles receive the lowest penalty (-1.0) since their positions are fixed. Adults incur a moderate penalty (-1.5) due to their predictable movement patterns. Cyclists warrant a higher penalty (-2.0) given their speed and fall risk. Collisions with children receive the maximum penalty (-2.5) as they are both unpredictable and highly vulnerable.

Let $d_{\text{disc}}(e)$ be the discomfort distance (in meters) for entity e:

$$d_{\text{disc}}(e) = \begin{cases} 0.1, & \text{if } e = A \\ 0.2, & \text{if } e = B \\ 0.2, & \text{if } e = C \end{cases} \tag{25}$$

起始目标抽样。在每个二元地图上，我们选择一到三个起始目标对。机器人的足迹被建模为 20 × 20 像素的正方形；有效的起始位置和目标位置是整个足迹完全位于自由空间内的位置。此外，我们要求目标距离起点足够远，以产生重要的情节：我们选择欧几里德距离至少为地图较大维度 0.75 的目标。为了确保全球层面的动态可行性，我们将障碍物膨胀了足迹厚度的一半，并在膨胀的网格上执行 8 连接的 A* 搜索（如第 4 节所述）。仅当存在无碰撞 A* 路径时才接受对；接受的路径与起点和目标一起保存。

结果数据集。由于我们的目标是训练机器人在足够复杂的环境中驾驶，因此我们通过引入最小静态占用率来丢弃没有或很少有静态障碍物的地图：障碍物百分比低于阈值（5%）的地图将被跳过和使用。没有有效的起始目标对满足路径存在约束的地图也将被删除。

在过滤和删除无法实现的情况后，我们获得了 30,948 个不同的地图。在每张地图上采样 1-3 个起始目标对，并保证从开始到结束的路径，会产生 77,663 个独特的导航片段。这些情节在我们的模拟器中充当静态占用地图。我们将它们随机分为训练集、验证集和测试集：对于我们的主要实验，我们使用 60,000 个集进行训练，500 个集用于验证，1,000 个集用于测试。剩余的剧集保留用于根据需要进行额外的实验。

5.2 奖励设置

在本小节中，我们定义实验的奖励函数。机器人在具有四种实体类型的环境中导航：成人、自行车、儿童和静态障碍物。每个动态实体类型有 4 个实例，因此每个场景有 12 个动态代理。静态障碍物由占用图定义，并因区域而异。

$$R_{\text{checkpoint}}(t) = \begin{cases} 0.3, & \text{if } \|\mathbf{p}_t - \mathbf{c}_k\|_2 < r_{\text{checkpoint}} \text{ and } k \text{ is unvisited checkpoint} \\ 0, & \text{otherwise} \end{cases} \tag{22}$$

$$R_{\text{success}} = 3 \tag{23}$$

我们将实体类型表示如下：成人 (A)、自行车 (B)、儿童 (C) 和障碍物 (O)。碰撞处罚定义为：

$$R_{\text{coll}}(e) = \begin{cases} -1.0, & \text{if } e = O \\ -1.5, & \text{if } e = A \\ -2.0, & \text{if } e = B \\ -2.5, & \text{if } e = C \end{cases} \tag{24}$$

我们根据不可预测性和潜在危害来调整碰撞处罚。静态障碍物受到的惩罚最低（-1.0），因为它们的位置是固定的。成年人由于其可预测的运动模式而受到中等惩罚（-1.5）。考虑到骑自行车者的速度和跌倒风险，他们应该受到更高的处罚（-2.0）。与儿童发生碰撞将受到最高处罚 (-2.5)，因为儿童难以预测且极易受到伤害。

令 $d_{\text{disc}}(e)$ 为实体 e 的不适距离（以米为单位）：

$$d_{\text{disc}}(e) = \begin{cases} 0.1, & \text{if } e = A \\ 0.2, & \text{if } e = B \\ 0.2, & \text{if } e = C \end{cases} \tag{25}$$

Let $p_{\text{disc}}(e)$ be the discomfort penalty factor for entity e:

$$p_{\text{disc}}(e) = \begin{cases} 0.0, & \text{if } e = O \\ 0.5, & \text{if } e = A \\ 1.0, & \text{if } e = B \\ 1.0, & \text{if } e = C \end{cases} \tag{26}$$

We omit $d_{\text{disc}}(O)$ because static obstacles cannot experience discomfort. However, if the robot should maintain distance from static obstacles, one can define $d_{\text{disc}}(O)$ and set $p_{\text{disc}}(O) > 0$.

5.3 Simulation Setup

We conduct experiments using our simulator designed for outdoor mobile robots operating in urban environments, such as sidewalk delivery robots, autonomous cleaners, and security patrol robots. The simulator models realistic pedestrian traffic and urban geometry based on the maps described in Section 5.1.

We compare three navigation methods: SARL [8], EB-CADRL [9], and our HMP-DRL.

Simulated agents (adults, children, bicyclists) are controlled by Optimal Reciprocal Collision Avoidance (ORCA) [11]. To introduce diversity, each agent's size and speed are drawn from a uniform distribution specific to its type. The distributions are designed to overlap, for instance, some children may be faster than some adults, making it impossible to determine an agent's type from its size and speed alone.

Dynamic agent spawning. In our simulator we use new dynamic agents during an episode ("dynamic spawn") to maintain traffic around the robot. In our setup, dynamic spawning is enabled and executed every 80 simulation steps (with $\Delta t = 0.25$ s, this corresponds to 20 s). At each spawn event, we add 4 adults, 4 bicyclists, and 4 children, and assign them a lifetime of 80 steps. Agents whose age exceeds the lifetime are removed from the scene. New agents are sampled from a square region of width 40 m, positioned and oriented relative to the robot: we compute the unit direction vector from the robot to its goal and place the square center ahead of the robot by half of the square width along this direction. The square yaw is aligned with the same robot-to-goal direction. Start and goal positions for spawned agents are sampled on opposite edges of this oriented square (square-crossing), with rejection sampling to avoid immediate collisions with the robot, other agents, and static obstacles.

All evaluations use an "invisible robot" setting, where other agents do not perceive or react to the robot. This tests the robot's collision avoidance capabilities in a worst-case scenario that is also realistic, as small robots are often overlooked by people.

Models are trained for 3000 episodes with imitation learning, followed by 60000 episodes using the algorithm "HMP-DRL Training with Parallel Deep V-learning" 2. During training, we validate the models every 1024 episodes on a 500-episode validation set. For the final comparison, we select the best-performing model for each method based on its validation performance using the weighted score metric and evaluate it on a 1,000-episode test set.

5.4 Results

Figure 4 shows training and validation curves.

All methods converge within 60,000 RL training episodes (after the imitation learning warm start).

Across both training and validation, HMP-DRL achieves consistently higher success rate and weighted score while maintaining lower collision rates than SARL and EB-CADRL.

To compare safety in a way that reflects different severity of collisions, we use the Weighted Score (WS) metric:

$$WS = SR - CR(A) - 4.0 \cdot CR(C) - 2.0 \cdot CR(B) - 0.5 \cdot CR(O). \tag{27}$$

令 $p_{\text{disc}}(e)$ 为实体 e 的不适惩罚因子：

$$p_{\text{disc}}(e) = \begin{cases} 0.0, & \text{if } e = O \\ 0.5, & \text{if } e = A \\ 1.0, & \text{if } e = B \\ 1.0, & \text{if } e = C \end{cases} \tag{26}$$

我们省略 $d_{\text{disc}}(O)$ 因为静态障碍物不会感到不适。然而，如果机器人需要与静态障碍物保持距离，则可以定义 $d_{\text{disc}}(O)$ 并将 $p_{\text{disc}}(O) >$ 设置为 0。

5.3 模拟设置

我们使用专为城市环境中运行的户外移动机器人设计的模拟器进行实验，例如人行道送货机器人、自动清洁工和安全巡逻机器人。模拟器根据第 5.1 节中描述的地图对现实的行人交通和城市几何形状进行建模。
我们比较了三种导航方法：SARL [8]、EB-CADRL [9] 和我们的 HMP-DRL。

模拟主体（成人、儿童、骑自行车的人）由最佳相互碰撞避免（ORCA）控制[11]。为了引入多样性，每个代理的大小和速度都是根据其类型特定的均匀分布得出的。这些分布被设计为重叠，例如，某些儿童可能比某些成人更快，因此无法仅根据代理的大小和速度来确定代理的类型。

动态代理生成。在我们的模拟器中，我们在一个情节（“ 动态生成 ”）期间使用新的动态代理来维持机器人周围的流量。在我们的设置中，每 80 个模拟步骤启用并执行动态生成（$\Delta t = 0.25$ 秒，这相当于 20 秒）。在每次生成事件中，我们添加 4 个成人、4 个骑自行车的人和 4 个儿童，并为他们分配 80 步的生命周期。年龄超过寿命的特工将被从现场移除。新代理从宽度为 40 m 的正方形区域中采样，相对于机器人进行定位和定向：我们计算从机器人到其目标的单位方向向量，并将正方形中心沿该方向放置在机器人前方正方形宽度的一半处。方形偏航与机器人到目标的相同方向对齐。生成的代理的起始位置和目标位置在此定向正方形（正方形交叉）的相对边缘上进行采样，并进行拒绝采样以避免与机器人、其他代理和静态障碍物立即发生碰撞。

所有评估都使用 “ 隐形机器人 ” 设置，其中其他代理不会感知机器人或对机器人做出反应。这测试了机器人在最坏情况下的避碰能力，这也是现实的，因为小型机器人经常被人们忽视。

使用模仿学习对模型进行 3000 集训练，然后使用 “ 并行深度 V 学习的 HMP-DRL 训练 ” 算法进行 60000 集训练。2. 在训练期间，我们在 500 集验证集上每 1024 集验证模型。为了进行最终比较，我们使用加权得分指标根据每种方法的验证性能选择性能最佳的模型，并在 1,000 集测试集上对其进行评估。

5.4 结果

图 4 显示了训练和验证曲线。

所有方法在 60,000 个 RL 训练集内收敛（在模仿学习热启动之后）。

在训练和验证过程中，HMP-DRL 始终实现更高的成功率和加权分数，同时保持比 SARL 和 EB-CADRL 更低的冲突率。
为了以反映不同碰撞严重程度的方式比较安全性，我们使用加权分数 (WS) 指标：

$$WS = SR - CR(A) - 4.0 \cdot CR(C) - 2.0 \cdot CR(B) - 0.5 \cdot CR(O). \tag{27}$$

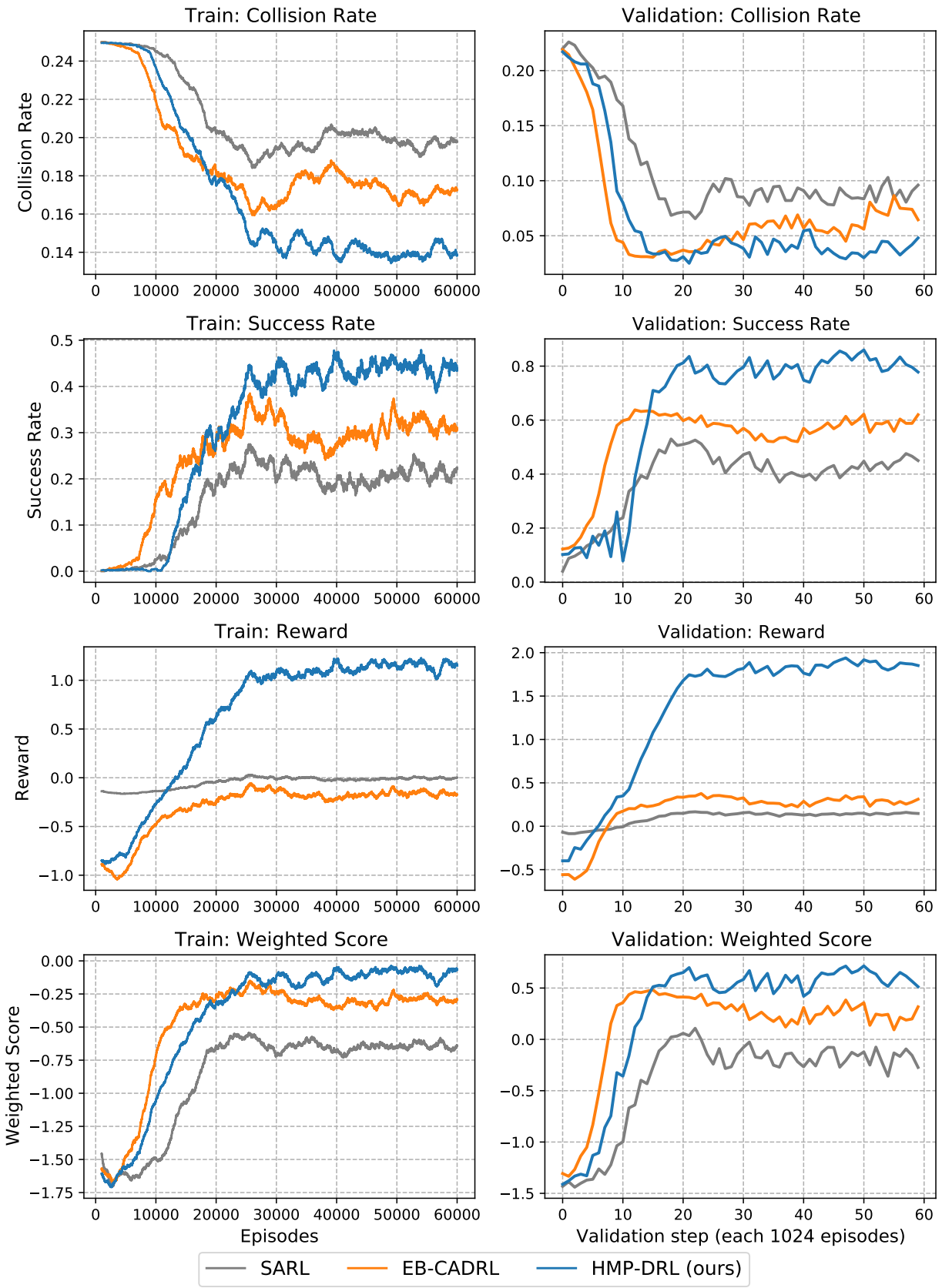

Figure 4: Collision rate, success rate, reward, weighted score metrics on the training and validation data. The validation step is performed every 1024 episodes.

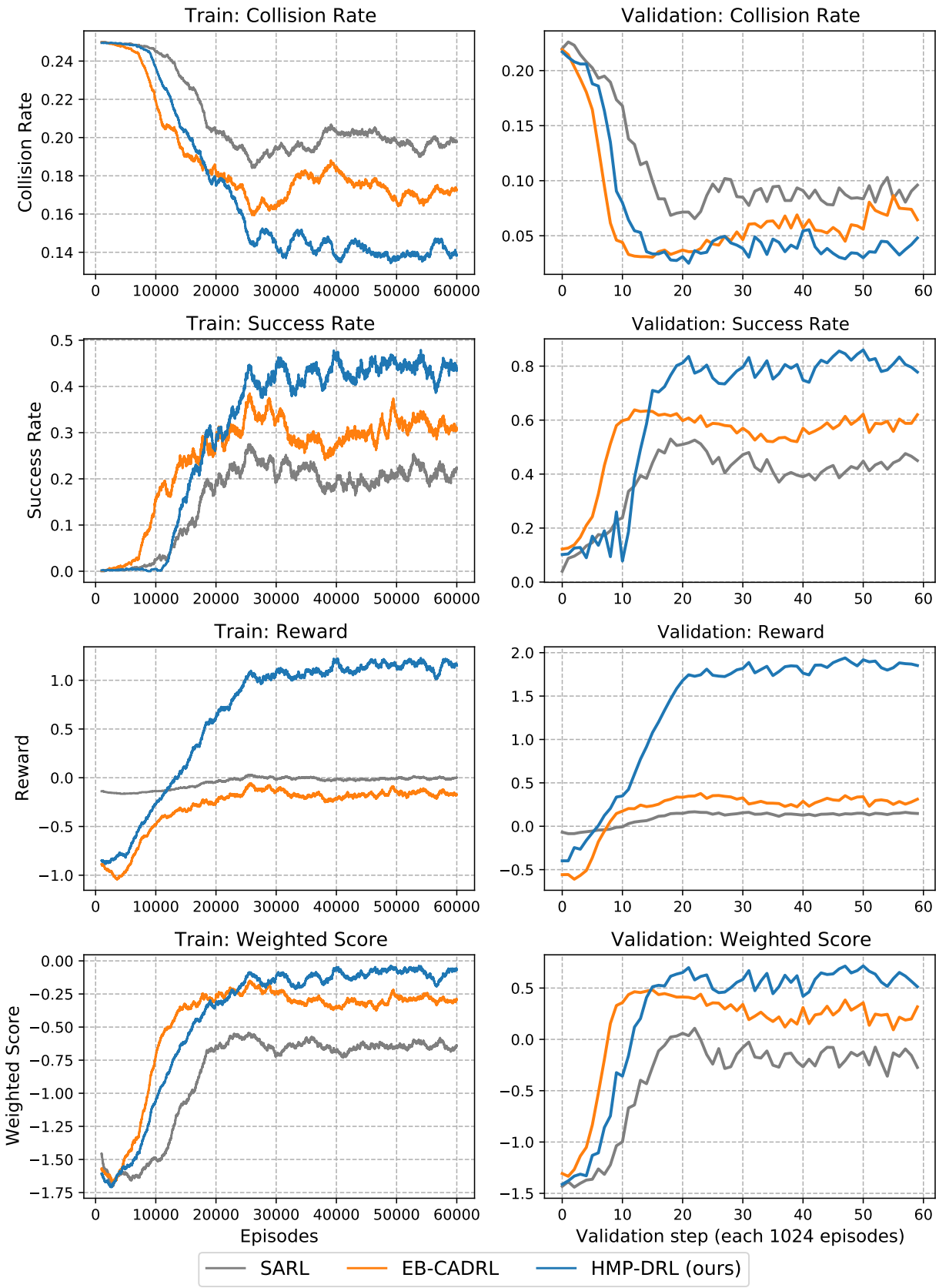

图 4：训练和验证数据的碰撞率、成功率、奖励、加权得分指标。每 1024 集执行一次验证步骤。

Table 1: Metrics on test data with 12 agents.

Method	SARL	EB-CADRL	HMP-DRL
SR	0.524	0.649	**0.836**
CR	0.289	0.128	**0.123**
CR(A)	0.05	**0.019**	**0.019**
CR(B)	0.155	0.047	**0.044**
CR(C)	0.022	0.015	**0.004**
CR(O)	0.062	**0.047**	0.056
Time	78.36	93.42	**72.03**
DD(A)	0.154	0.147	**0.166**
DD(B)	0.144	0.181	**0.183**
DD(C)	0.153	0.184	**0.209**
WS	0.045	0.4525	**0.685**

Table 2: Metrics on test data with 3 agents (1 adult, 1 bicycle, 1 child).

Method	SARL	EB-CADRL	HMP-DRL
SR	0.719	0.712	**0.936**
CR	0.054	0.034	**0.017**
CR(A)	0.009	0.003	**0.001**
CR(B)	0.018	0.015	**0.007**
CR(C)	0.004	**0.0**	**0.0**
CR(O)	0.023	0.016	**0.009**
Time	89.72	98.89	**72.84**
DD(A)	0.15	0.166	**0.182**
DD(B)	0.151	**0.169**	0.15
DD(C)	0.157	**0.249**	0.208
WS	0.6465	0.671	**0.9165**

Here, SR is the success rate and $CR(\cdot)$ is the collision rate with adults (A), children (C), bicycles (B), and static obstacles (O). Coefficients are chosen to penalize collisions with more vulnerable entities more heavily and can be tuned to match application-specific safety priorities.

Since each method uses a different reward definition, episode returns are not directly comparable across methods and are used only to monitor within-method learning progress.

Table 1 summarizes evaluation on the 1,000-episode test set. We report: *SR* (success rate); *CR* (overall collision rate); *CR(A), CR(B), CR(C), CR(O)* (collision rates with adults, bicycles, children, and static obstacles); *Time* (average time to reach the goal); and *DD(A), DD(B), DD(C)* (mean robot–agent distance during "danger" interactions for adults, bicycles, and children, where danger is defined as distance < 30 cm).

HMP-DRL achieves a substantially higher success rate (0.836) than EB-CADRL (0.649) and SARL (0.524), while also reducing the overall collision rate (0.123 vs. 0.128 and 0.289). The largest safety gain is in collisions with children: $CR(C)$ drops to 0.004 for HMP-DRL (vs. 0.015 for EB-CADRL and 0.022 for SARL), which strongly improves the weighted score (0.685 vs. 0.4525 and 0.045). HMP-DRL also reaches the goal faster on average (72.03 s vs. 93.42 s and 78.36 s).

Table 2 reports results in a simpler scenario with 3 dynamic agents (1 adult, 1 bicycle, 1 child), which reduces the influence of dense surrounding traffic. In this setting, HMP-DRL further improves performance, achieving $SR = 0.936$ and $CR = 0.017$, with near-zero collisions with vulnerable agents ($CR(A) = 0.001$, $CR(C) = 0.0$). Consequently, it attains the highest weighted score (0.9165) and the lowest average time to reach the goal (72.84 s).

Overall, HMP-DRL performs best on the main metrics: it achieves the highest success rate (SR) and weighted score (WS), and the lowest collision rate (CR), while maintaining comparable safety margins during close interactions.

表 1：12 种代理的测试数据指标。

Method	SARL	EB-CADRL	HMP-DRL
SR	0.524	0.649	**0.836**
CR	0.289	0.128	**0.123**
CR(A)	0.05	**0.019**	**0.019**
CR(B)	0.155	0.047	**0.044**
CR(C)	0.022	0.015	**0.004**
CR(O)	0.062	**0.047**	0.056
Time	78.36	93.42	**72.03**
DD(A)	0.154	0.147	**0.166**
DD(B)	0.144	0.181	**0.183**
DD(C)	0.153	0.184	**0.209**
WS	0.045	0.4525	**0.685**

表 2：3 名代理人（1 名成人、1 辆自行车、1 名儿童）的测试数据指标。

Method	SARL	EB-CADRL	HMP-DRL
SR	0.719	0.712	**0.936**
CR	0.054	0.034	**0.017**
CR(A)	0.009	0.003	**0.001**
CR(B)	0.018	0.015	**0.007**
CR(C)	0.004	**0.0**	**0.0**
CR(O)	0.023	0.016	**0.009**
Time	89.72	98.89	**72.84**
DD(A)	0.15	0.166	**0.182**
DD(B)	0.151	**0.169**	0.15
DD(C)	0.157	**0.249**	0.208
WS	0.6465	0.671	**0.9165**

这里，SR是成功率，$CR(\cdot)$是与成人(A)、儿童(C)、自行车(B)和静态障碍物(O)的碰撞率。选择系数来更严厉地惩罚与更脆弱实体的碰撞，并且可以进行调整以匹配特定于应用程序的安全优先级。

由于每种方法使用不同的奖励定义，因此不同方法之间的情节回报不能直接比较，并且仅用于监控方法内的学习进度。
表 1 总结了 1,000 集测试集的评估。我们报告：*SR* (成功率)；*CR* (总体碰撞率); *CR(A), CR(B), CR(C), CR(O)* (与成人、自行车、儿童和静态障碍物的碰撞率)；*Time* (达到目标的平均时间)；*DD(A), DD(B), DD(C)* (表示成人、自行车和儿童在“危险”交互期间机器人与代理的距离，其中危险定义为距离 $<$ 30 cm)。
HMP-DRL 的成功率 (0.836) 明显高于 EB-CADRL (0.649) 和 SARL (0.524)，同时还降低了总体碰撞率（0.123 vs. 0.128 和 0.289）。最大的安全增益是在与儿童发生碰撞时：HMP-DRL 的 $CR(C)$ 下降至 0.004（而 EB-CADRL 为 0.015，SARL 为 0.022），这大大提高了加权分数（0.685 对比 0.452 5 和 0.045）。 HMP-DRL 平均达到目标的速度也更快（72.03 秒对比 93.42 秒和 78.36 秒）。

表 2 报告了具有 3 个动态代理（1 名成人、1 名自行车、1 名儿童）的更简单场景的结果，这减少了周围密集交通的影响。在此设置中，HMP-DRL 进一步提高了性能，实现了 $SR = 0.936$ 和 $CR = 0.017$，与易受攻击的代理的碰撞几乎为零（$CR(A) = 0.001$、$CR(C) = 0.0$）。因此，它获得了最高的加权分数（0.9165）和最短的平均到达目标时间（72.84 秒）。

总体而言，HMP-DRL 在主要指标上表现最佳：它实现了最高的成功率（SR）和加权得分（WS）以及最低的碰撞率（CR），同时在密切交互过程中保持了相当的安全裕度。

5.5 Ablation Experiments

Table 3: Ablation study: effect of checkpoint reward shaping (R_{cp}) on HMP-DRL.

Method	With checkpoint reward ($R_{\text{cp}} = 0.3$)	Without checkpoint reward ($R_{\text{cp}} = 0.0$)
SR	**0.836**	0.615
CR	**0.123**	0.13
CR(A)	0.019	**0.018**
CR(B)	**0.044**	0.054
CR(C)	**0.004**	0.008
CR(O)	0.056	**0.05**
Time	**72.03**	79.59
DD(A)	**0.166**	0.153
DD(B)	**0.183**	0.182
DD(C)	**0.209**	0.2
WS	**0.685**	0.432

We ablate the checkpoint-based reward shaping term (Section 4.3, Eq. (11)), which provides intermediate progress feedback along the global A* path via checkpoints. We train two HMP-DRL policies from scratch with identical architecture, datasets, and hyperparameters; the only difference is the checkpoint reward magnitude R_{cp}. In the first variant we use the default setting $R_{\text{cp}} = 0.3$, while in the second we disable checkpoint shaping by setting $R_{\text{cp}} = 0.0$. Both policies are evaluated on the same 1,000-episode test set as in Table 1, and we report the same metrics.

The results are summarized in Table 3. Enabling checkpoint rewards substantially improves navigation performance: the success rate increases from 0.615 to 0.836 and the weighted score increases from 0.432 to 0.685. The policy also reaches the goal faster on average (72.03 s vs. 79.59 s), indicating more efficient progress and fewer stalled trajectories. Safety improves primarily for the most challenging dynamic entities: collisions with bicycles decrease (0.054 $\rightarrow$ 0.044) and collisions with children are halved (0.008 $\rightarrow$ 0.004). We also observe larger mean danger distances $DD(\cdot)$, suggesting the checkpoint-shaped policy maintains slightly larger safety margins during close interactions. While $CR(A)$ and $CR(O)$ change only marginally (and in opposite directions), the overall effect of checkpoint shaping is strongly positive across the key metrics.

These findings support our design choice: sparse checkpoint feedback aligns the local policy with the global plan, improving all main metrics.

6 Conclusion

In this work, we proposed *Hybrid Motion Planning with Deep Reinforcement Learning* (HMP-DRL), a hybrid navigation approach that couples a graph-based global planner with a local DRL policy. The global level computes global path and converts it into a sequence of distance-spaced checkpoints. The local level extends an attention-based, entity-aware value network with checkpoint features and trains it with reward shaping that encourages progress along the route while enforcing type-dependent safety margins and collision penalties for different entities.

Experiments in a large-scale urban simulator derived from OpenStreetMap, with dense ORCA-controlled traffic, show that HMP-DRL improves both reliability and safety. On the main test set, it achieves a success rate of 0.836 (vs. 0.649 for EB-CADRL and 0.524 for SARL) with a lower overall collision rate of 0.123 and faster traversal time (72.03 s on average). The largest gain is for the most vulnerable entity type: collisions with children drop to 0.004, yielding the best weighted safety score (0.685). An ablation study further confirms the role of route guidance: disabling checkpoint rewards reduces the success rate from 0.836 to 0.615 and the weighted score from 0.685 to 0.432.

5.5 消融实验

表 3：消融研究：检查点奖励塑造 (R_{cp}) 对 HMP-DRL 的影响。

Method	With checkpoint reward ($R_{\text{cp}} = 0.3$)	Without checkpoint reward ($R_{\text{cp}} = 0.0$)
SR	**0.836**	0.615
CR	**0.123**	0.13
CR(A)	0.019	**0.018**
CR(B)	**0.044**	0.054
CR(C)	**0.004**	0.008
CR(O)	0.056	**0.05**
Time	**72.03**	79.59
DD(A)	**0.166**	0.153
DD(B)	**0.183**	0.182
DD(C)	**0.209**	0.2
WS	**0.685**	0.432

我们取消了基于检查点的奖励塑造项（第 4.3 节，方程（11）），它通过检查点沿着全局 A* 路径提供中间进度反馈。我们使用相同的架构、数据集和超参数从头开始训练两个 HMP-DRL 策略；唯一的区别是检查点奖励大小 R_{cp}。在第一个变体中，我们使用默认设置 $R_{\text{cp}} = 0.3$，而在第二个变体中，我们通过设置 $R_{\text{cp}} = 0.0$ 来禁用检查点整形。这两种策略都在表 1 中相同的 1,000 集测试集上进行评估，并且我们报告了相同的指标。

结果总结在表 3 中。启用检查点奖励显着提高了导航性能：成功率从 0.615 增加到 0.836，加权分数从 0.432 增加到 0.685。该策略平均达到目标的速度也更快（72.03 秒 vs. 79.59 秒），这表明进展更有效，停滞轨迹更少。安全性的提高主要针对最具挑战性的动态实体：与自行车的碰撞减少（0.054 → 0.044），与儿童的碰撞减少一半（0.008 → 0.004）。我们还观察到更大的平均危险距离 $DD(\cdot)$，这表明检查点形状的政策在密切互动期间保持了稍大的安全边际。虽然 $CR(A)$ 和 $CR(O)$ 仅略有变化（并且方向相反），但检查点整形的总体效果在关键指标上具有强烈的积极作用。

这些发现支持我们的设计选择：稀疏的检查点反馈使本地政策与全球计划保持一致，从而改善所有主要指标。

6 结论

在这项工作中，我们提出了 *Hybrid Motion Planning with Deep Reinforcement Learning* (HMP-DRL)，这是一种混合导航方法，它将基于图的全局规划器与本地 DRL 策略结合起来。全局级别计算全局路径并将其转换为一系列间隔距离的检查点。本地级别扩展了一个具有检查点功能的基于注意力的实体感知价值网络，并通过奖励塑造对其进行训练，以鼓励沿路线取得进展，同时对不同实体实施与类型相关的安全裕度和碰撞惩罚。

在源自 OpenStreetMap 的大型城市模拟器（具有 ORCA 控制的密集交通）中进行的实验表明，HMP-DRL 提高了可靠性和安全性。在主要测试集上，它实现了 0.836 的成功率（相对于 EB-CADRL 的 0.649 和 SARL 的 0.524），总体碰撞率较低，为 0.123，遍历时间更快（平均 72.03 秒）。最大的增益是针对最脆弱的实体类型：与儿童的碰撞下降至 0.004，产生最佳加权安全得分 (0.685)。消融研究进一步证实了路线引导的作用：禁用检查点奖励会将成功率从 0.836 降低到 0.615，加权分数从 0.685 降低到 0.432。

Future research directions include integrating dynamic occupancy maps with online graph re-planning, adding additional entity types (e.g., cars and motorcycles), training policies for complex scenarios—such as road crossings with dynamic vehicles, and real-world testing.

References

[1] Roland Siegwart, Illah R. Nourbakhsh, and Davide Scaramuzza. *Introduction to Autonomous Mobile Robots*. The MIT Press, 2nd edition, 2011. ISBN 0262015358.

[2] Giuseppe Fragapane, René de Koster, Fabio Sgarbossa, and Jan Ola Strandhagen. Planning and control of autonomous mobile robots for intralogistics: Literature review and research agenda. *European Journal of Operational Research*, 294(2):405–426, 2021. ISSN 0377-2217. doi:https://doi.org/10.1016/j.ejor.2021.01.019. URL `https://www.sciencedirect.com/science/article/pii/S0377221721000217`.

[3] George A. Bekey. *Autonomous Robots: From Biological Inspiration to Implementation and Control*. The MIT Press, 2005. ISBN 0262025787.

[4] Daniel S Drew. Multi-agent systems for search and rescue applications. *Current Robotics Reports*, 2: 189–200, 2021. doi:10.1007/s43154-021-00048-3.

[5] Thibault Kruse, Amit Kumar Pandey, Rachid Alami, and Alexandra Kirsch. Human-aware robot navigation: A survey. *Robotics and Autonomous Systems*, 61(12):1726–1743, 2013. ISSN 0921-8890.

[6] Steven R. Gehrke, Christopher D. Phair, Brendan J. Russo, and Edward J. Smaglik. Observed sidewalk autonomous delivery robot interactions with pedestrians and bicyclists. *Transportation Research Interdisciplinary Perspectives*, 18:100789, 2023. ISSN 2590-1982.

[7] Yu Fan Chen, Miao Liu, Michael Everett, and Jonathan P. How. Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, page 285–292, 2017.

[8] Changan Chen, Yuejiang Liu, Sven Kreiss, and Alexandre Alahi. Crowd-robot interaction: Crowd-aware robot navigation with attention-based deep reinforcement learning. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 6015–6022, 2019.

[9] Yury Kolomeytsev and Dmitry Golembiovsky. Robot navigation with entity-based collision avoidance using deep reinforcement learning, 2025. URL `https://arxiv.org/abs/2408.14183`.

[10] Dirk Helbing and Péter Molnár. Social force model for pedestrian dynamics. *Phys. Rev. E*, 51: 4282–4286, May 1995. doi:10.1103/PhysRevE.51.4282.

[11] Jur van den Berg, Stephen J. Guy, Ming Lin, and Dinesh Manocha. Reciprocal n-body collision avoidance. In Cédric Pradalier, Roland Siegwart, and Gerhard Hirzinger, editors, *Robotics Research*, pages 3–19, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg. ISBN 978-3-642-19457-3.

[12] Lei Tai, Jingwei Zhang, Ming Liu, and Wolfram Burgard. Socially compliant navigation through raw depth inputs with generative adversarial imitation learning, 05 2018.

[13] Henrik Kretzschmar, Markus Spies, Christoph Sprunk, and Wolfram Burgard. Socially compliant mobile robot navigation via inverse reinforcement learning. *The International Journal of Robotics Research*, 35, 01 2016. doi:10.1177/0278364915619772.

[14] Yu Fan Chen, Michael Everett, Miao Liu, and Jonathan P. How. Socially aware motion planning with deep reinforcement learning. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1343–1350, 2017.

[15] Michael Everett, Yu Fan Chen, and Jonathan P. How. Motion planning among dynamic, decision-making agents with deep reinforcement learning. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 3052–3059, 2018.

[16] Bingxin Xue et al. Crowd-aware socially compliant robot navigation via deep reinforcement learning. *International Journal of Social Robotics*, 16(1):197–209, 2024. ISSN 1875-4805.

未来的研究方向包括将动态占用地图与在线图形重新规划相结合、添加额外的实体类型（例如汽车和摩托车）、复杂场景的训练策略（例如动态车辆的道路交叉口）以及真实世界的测试。

参考

[1] Roland Siegwart、Illah R. Nourbakhsh 和 Davide Scaramuzza。 *Introduction to Autonomous Mobile Robots*。麻省理工学院出版社，第 2 版，2011 年。ISBN 0262015358。[2] Giuseppe Fragapane、René de Koster、Fabio Sgarbossa 和 Jan Ola Strandhagen。内部物流自主移动机器人的规划和控制：文献综述和研究议程。 *European Journal of Operational Research*，294(2)：405 – 426，2021。ISSN 0377-2217。 doi：https://doi.org/10.1016/j.ejor.2021.01.019。网址 https://www.sciencedirect.com/science/article/pii/S0377221721000217。 [3] 乔治·A·贝基。
Autonomous Robots: From Biological Inspiration to Implementation and Control。麻省理工学院出版社，2005 年。ISBN 0262025787。[4] Daniel S Drew。用于搜索和救援应用的多代理系统。
Current Robotics Reports，2：189 – 200，2021。doi：10.1007/s43154-021-00048-3。 [5] Thibault Kruse、Amit Kumar Pandey、Rachid Alami 和 Alexandra Kirsch。人类感知机器人导航：一项调查。
Robotics and Autonomous Systems，61(12)：1726 – 1743，2013。ISSN 0921-8890。 [6] 史蒂文·R·格尔克、克里斯托弗·D·菲尔、布伦丹·J·鲁索和爱德华·J·斯马格利克。观察人行道自主送货机器人与行人和骑自行车者的互动。 *Transportation Research Interdisciplinary Perspectives*，18:100789，2023。ISSN 2590-1982。 [7] 陈宇凡、刘淼、Michael Everett、Jonathan P. How。通过深度强化学习来避免分散的非通信多智能体碰撞。在 *2017 IEEE International Conference on Robotics and Automation (ICRA)* 中，第 285 – 292 页，2017 年。 [8] Changan Chen、Yuejian Liu、Sven Kreiss 和 Alexandre Alahi。人群与机器人交互：具有基于注意力的深度强化学习的人群感知机器人导航。载于
2019 International Conference on Robotics and Automation (ICRA)，第 6015 – 6022 页，2019 年。 [9] Yury Kolomeytsev 和 Dmitry Golembiovsky。使用深度强化学习实现基于实体的防撞机器人导航，2025 年。URL https://arxiv.org/abs/2408.14183。 [10] 德克·赫尔宾和彼得·莫纳尔。行人动力学的社会力模型。 *Phys. Rev. E*，51：4282 – 4286，1995 年 5 月。doi：10.1103/PhysRevE.51.4282。 [11] Jur van den Berg、Stephen J. Guy、Ming Lin 和 Dinesh Manocha。相互的 n 体碰撞避免。 Cédric Pradalier、Roland Siegwart 和 Gerhard Hirzinger，编辑，*Robotics Research*，第 3-19 页，柏林，海德堡，2011 年。施普林格柏林海德堡。 ISBN 978-3-642-19457-3。 [12] 泰雷，张经纬，刘明，Wolfram Burgard。通过生成对抗性模仿学习通过原始深度输入实现社交合规导航，2018 年 5 月。 [13] Henrik Kretzschmar、Markus Spies、Christoph Sprunk 和 Wolfram Burgard。通过逆强化学习实现符合社交要求的移动机器人导航。 *The International Journal of Robotics Research*, 35, 01 2016.doi:10.1177/0278364915619772。 [14] 陈宇凡、Michael Everett、Miao Liu 和 Jonathan P. How。具有深度强化学习的社会意识运动规划。载于 *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*，第 1343 – 1350 页，2017 年。 [15] Michael Everett、Yu Fan Chen 和 Jonathan P. How。通过深度强化学习在动态决策代理之间进行运动规划。 In
2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)，第 3052 – 3059 页，2018 年。 [16] Bingxin Xu 等人。通过深度强化学习实现群体感知、社交合规的机器人导航。
International Journal of Social Robotics，16(1)：197 – 209，2024。ISSN 1875-4805。

[17] Keyu Li, Yangxin Xu, Jiankun Wang, and Max Q.-H. Meng. Sarl: Deep reinforcement learning based human-aware navigation for mobile robot in indoor environments. In *2019 IEEE International Conference on Robotics and Biomimetics (ROBIO)*, pages 688–694, 2019. doi:10.1109/ROBIO49542.2019.8961764.

[18] Binyu Wang, Zhe Liu, Qingbiao Li, and Amanda Prorok. Mobile robot path planning in dynamic environments through globally guided reinforcement learning. *IEEE Robotics and Automation Letters*, 5(4):6932–6939, 2020. doi:10.1109/LRA.2020.3026638.

[19] Ekim Yurtsever, Linda Capito, Keith Redmill, and Umit Ozguner. Integrating deep reinforcement learning with model-based path planners for automated driving. In *2020 IEEE Intelligent Vehicles Symposium (IV)*, pages 1311–1316. IEEE, 2020.

[20] Reinis Cimurs, Il Hong Suh, and Jin Han Lee. Goal-driven autonomous exploration through deep reinforcement learning. *IEEE Robotics and Automation Letters*, 7(2):730–737, 2022. doi:10.1109/LRA.2021.3133591.

[21] Yunfei Bi and Xi Fang. A hybrid path planning framework integrating deep reinforcement learning and variable-direction potential fields. *Mathematics*, 13(14), 2025. ISSN 2227-7390. doi:10.3390/math13142312. URL `https://www.mdpi.com/2227-7390/13/14/2312`.

[22] X. Zhong, J. Tian, H. Hu, et al. Hybrid path planning based on safe a* algorithm and adaptive window approach for mobile robot in large-scale dynamic environment. *Journal of Intelligent & Robotic Systems*, 99:65–77, 2020. doi:10.1007/s10846-019-01112-z.

[23] Y. Sun, Q. Yuan, Q. Gao, et al. A multiple environment available path planning based on an improved a* algorithm. *International Journal of Computational Intelligence Systems*, 17:172, 2024. doi:10.1007/s44196-024-00571-z.

[24] Youngmi Baek and Jung Kyu Park. Fast path generation algorithm for mobile robot navigation using hybrid map. *Applied Sciences*, 15(5), 2025. ISSN 2076-3417. doi:10.3390/app15052414. URL `https://www.mdpi.com/2076-3417/15/5/2414`.

[25] Hoang-Long Pham, Nhu-Nghia Bui, and Thai-Viet Dang. Hybrid path planning for wheeled mobile robot based on rrt-star algorithm and reinforcement learning method. *Journal of Robotics and Control (JRC)*, 6(4):2045–2051, 2025. doi:10.18196/jrc.v6i4.27678.

[26] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. A Bradford Book, Cambridge, MA, USA, 2018. ISBN 0262039249.

[27] Jean-Claude Latombe. *Robot Motion Planning*. Kluwer Academic Publishers, Norwell, MA, United States, 1991. ISBN 978-0-7923-9129-6.

[28] Marcelo Kallmann. Path planning in triangulations. In *Proceedings of the IJCAI Workshop on Reasoning, Representation, and Learning in Computer Games*, pages 49–54, 2005.

[29] Ahmadreza Meysami, Jean-Christophe Cuillière, Vincent François, and Sousso Kelouwani. Investigating the impact of triangle and quadrangle mesh representations on agv path planning for various indoor environments: With or without inflation. *Robotics*, 11(2):50, 2022. doi:10.3390/robotics11020050.

[30] H. Choset and J. Burdick. Sensor based planning. i. the generalized voronoi graph. In *Proceedings of 1995 IEEE International Conference on Robotics and Automation*, volume 2, pages 1649–1655 vol.2, 1995. doi:10.1109/ROBOT.1995.525511.

[31] Franz Aurenhammer. Voronoi diagrams—a survey of a fundamental geometric data structure. *ACM Comput. Surv.*, 23(3):345–405, September 1991. ISSN 0360-0300. doi:10.1145/116873.116880. URL `https://doi.org/10.1145/116873.116880`.

[32] Tomás Lozano-Pérez and Michael A. Wesley. An algorithm for planning collision-free paths among polyhedral obstacles. *Commun. ACM*, 22(10):560–570, October 1979. ISSN 0001-0782. doi:10.1145/359156.359164. URL `https://doi.org/10.1145/359156.359164`.

[33] Junlin Ou, Seong Hyeon Hong, Ge Song, and Yi Wang. Hybrid path planning based on adaptive visibility graph initialization and edge computing for mobile robots. *Engineering Applications of Artificial Intelligence*, 126:107110, 2023. ISSN 0952-1976. doi:https://doi.org/10.1016/j.engappai.2023.107110. URL `https://www.sciencedirect.com/science/article/pii/S0952197623012940`.

[17] 李克宇，徐阳欣，王建坤，Max Q.-H。孟。 Sarl：基于深度强化学习的人类感知导航，用于室内环境中的移动机器人。在 *2019 IEEE International Conference on Robotics and Biomimetics (ROBIO)* 中，第 688 – 694 页，2019 年。doi:10.1109/ROBIO49542.2019.8961764。 [18]王斌宇，刘哲，李清彪，Amanda Prorok。通过全局引导强化学习在动态环境中进行移动机器人路径规划。 *IEEE Robotics and Automation Letters*，5(4)：6932 – 6939，2020。doi：10.1109/LRA.2020.3026638。 [19] Ekim Yurtsever、Linda Capito、Keith Redmill 和 Umit Ozguner。将深度强化学习与基于模型的路径规划器集成以实现自动驾驶。在 *2020 IEEE Intelligent Vehicles Symposium (IV)* 中，第 1311 – 1316 页。 IEEE，2020。[20] Reinis Cimurs、Il Hong Suh 和 Jin Han Lee。通过深度强化学习实现目标驱动的自主探索。 *IEEE Robotics and Automation Letters*，7(2)：730 – 737，2022。doi：10.1109/LRA.2021.3133591。 [21] 毕云飞，方曦.集成深度强化学习和可变方向势场的混合路径规划框架。 *Mathematics*，13(14)，2025。ISSN 2227-7390。 doi：10.3390/math13142312。网址 https://www.mdpi.com/2227-7390/13/14/2312。 [22] 钟旭，田建，胡红，等。大规模动态环境下移动机器人基于safe a*算法和自适应窗口方法的混合路径规划*Journal of Intelligent & Robotic Systems*，99:65 – 77，2020。doi:10.1007/s10846-019-01112-z。 [23] 孙勇，袁强，高强，等。基于改进的a*算法的多环境可用路径规划。 *International Journal of Computational Intelligence Systems*，17:172，2024。doi:10.1007/s44196-024-00571-z。 [24] Youngmi Baek 和 Jung Kyu Park。使用混合地图的移动机器人导航快速路径生成算法 *Applied Sciences*，15(5)，2025。ISSN 2076-3417。 doi：10.3390/app15052414。网址 https://www.mdpi.com/2076-3417/15/5/2414。 [25] Hoang-Long Pham、Nhu-Nghia Bui 和 Thai-Viet Dang。基于rrt-star算法和强化学习方法的轮式移动机器人混合路径规划*Journal of Robotics and Control (JRC)*, 6(4):2045 – 2051, 2025.doi:10.18196/jrc.v6i4.27678。 [26] 理查德·S·萨顿和安德鲁·G·巴托。 *Reinforcement Learning: An Introduction*。布拉德福德书籍，美国马萨诸塞州剑桥，2018 年。ISBN 0262039249。[27] Jean-Claude Latombe。 *Robot Motion Planning*。 Kluwer 学术出版社，美国马萨诸塞州诺威尔，1991 年。ISBN 978-0-7923-9129-6。 [28] 马塞洛·卡尔曼。三角测量中的路径规划。 *Proceedings of the IJCAI Workshop on Reasoning, Representation, and Learning in Computer Games*，第 49-54 页，2005 年。 [29] Ahmadreza Meysami、Jean-Christophe Cuillière、Vincent François 和 Sousso Kelouwani。研究三角形和四边形网格表示对各种室内环境的 AGV 路径规划的影响：有或没有充气。 *Robotics*，11(2):50，2022。doi:10.3390/robotics11020050。 [30] H. Choset 和 J. Burdick。基于传感器的规划。我。广义 voronoi 图。在 *Proceedings of 1995 IEEE International Conference on Robotics and Automation*，第 2 卷，第 1649 – 1655 页，第 2 卷，1995 年。doi:10.1109/ROBOT.1995.525511。 [31] 弗朗茨·奥伦哈默。沃罗诺伊图——对基本几何数据结构的调查。 *ACM Comput. Surv.*，23(3):345 – 405，1991 年 9 月。ISSN 0360-0300。号码：10.1145/116873.116880。网址 https://doi.org/10.1145/116873.116880。 [32] 托马斯·洛萨诺-佩雷斯和迈克尔·A·韦斯利。一种规划多面体障碍物之间无碰撞路径的算法。 *Commun. ACM*，22(10)：560 – 570，1979 年 10 月。ISSN 0001-0782。号码：10.1145/359156.359164。网址 https://doi.org/10.1145/359156.359164。 [33] 欧俊林，洪成贤，宋歌，王毅。基于自适应可见图初始化和边缘计算的移动机器人混合路径规划。 *Engineering Applications of Artificial Intelligence*，126：107110，2023。ISSN 0952-1976。 doi：https://doi.org/10.1016/j.engappai.2023.107110。网址 https://www.sciencedirect.com/science/article/pii/S0952197623012940。

[34] Peter Hart, Nils Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. doi:10.1109/tssc.1968.300136.

[35] Edsger W Dijkstra. A note on two problems in connexion with graphs. *Numerische mathematik*, 1(1): 269–271, 1959.

[36] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms, Third Edition*. The MIT Press, 3rd edition, 2009. ISBN 0262033844.

[37] Anthony (Tony) Stentz. The d* algorithm for real-time planning of optimal traverses. Technical Report CMU-RI-TR-94-37, Pittsburgh, PA, October 1994.

[38] Steven M. Lavalle. *Planning Algorithms*. Cambridge University Press, 2006. ISBN 0521862051.

[39] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In Francis Bach and David Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 448–456, Lille, France, 07–09 Jul 2015. PMLR. URL `https://proceedings.mlr.press/v37/ioffe15.html`.

[40] Adam Paszke et al. Pytorch: an imperative style, high-performance deep learning library. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, Red Hook, NY, USA, 2019. Curran Associates Inc.

[41] Bruno Siciliano and Oussama Khatib. *Springer Handbook of Robotics*. Springer Handbooks. Springer, 2016. ISBN 978-3-319-32550-7. doi:10.1007/978-3-319-32552-1.

[42] OpenStreetMap contributors. Planet dump retrieved from https://planet.osm.org . `https://www.openstreetmap.org`, 2017.

[34] 彼得·哈特、尼尔斯·尼尔森和伯特伦·拉斐尔。启发式确定最小成本路径的正式基础。 *IEEE Transactions on Systems Science and Cybernetics*，4(2)：100 – 107，1968。doi：10.1109/tssc.1968.300136。 [35] 埃兹格·W·迪杰斯特拉。关于与图表相关的两个问题的说明。 *Numerische mathematik*，1(1)：269 – 271，1959。 [36]托马斯·H·科门、查尔斯·E·莱瑟森、罗纳德·L·里维斯特和克利福德·斯坦。 *Introduction toAlgorithms, Third Edition*。麻省理工学院出版社，第 3 版，2009 年。ISBN 0262033844。 [37] 安东尼（托尼）斯坦茨。用于实时规划最佳遍历的 d* 算法。技术报告 CMU-RI-TR-94-37，匹兹堡，宾夕法尼亚州，1994 年 10 月。 [38] Steven M. Lavalle。 *Planning Algorithms*。剑桥大学出版社，2006。ISBN 0521862051。[39] Sergey Ioffe 和 Christian Szegedy。批量归一化：通过减少内部协变量偏移来加速深度网络训练。载于 Francis Bach 和 David Blei，编辑，*Proceedings of the 32nd International Conference on Machine Learning*，*Proceedings of Machine Learning Research* 第 37 卷，第 448-456 页，法国里尔，2015 年 7 月 7-09 日。PMLR。网址 https://proceedings.mlr。按/v37/ioffe15.html。 [40] 亚当·帕斯克等人。 Pytorch：命令式高性能深度学习库。 *Proceedings of the 33rd International Conference on Neural Information Processing Systems*，美国纽约州雷德胡克，2019 年。Curran Associates Inc. [41] Bruno Siciliano 和 Oussama Khatib。 *Springer Handbook of Robotics*。施普林格手册。施普林格，2016。ISBN 978-3-319-32550-7。 doi：10.1007/978-3-319-32552-1。 [42] OpenStreetMap 贡献者。从 https://planet.osm.org 检索的行星转储。 https://www. openstreetmap.org，2017。